

Freenet User Manual

Everything you need to install, use, check, fix, and grow with your Freenet node.

Manual revision	**1.19**
Written against Freenet	**v0.2.156**
Date	**2026-11-09**
Source	`docs/user-manual/` in freenet-core

Contents

About This Manual

Part I — Understanding Freenet

1. What Freenet Is

Part II — Getting Started

2. Installing Freenet
3. First Run and the Dashboard UPDATED
4. Using Freenet Applications

Part III — Operating Your Node

5. The Service: Day-to-Day Control
6. Configuration

Part IV — The Operational Self-Check

7. The Ten-Step Health Check
8. Reading the Signals
9. Correct what needs correcting — the principle
10. Diagnostic Reports

Part V — Correcting Problems

11. Troubleshooting by Symptom
12. Getting Help

Part VI — Upgrading

13. How Auto-Update Works
14. Updating Manually
15. Upgrading Yourself: as you advance

Part VII — Advancing: Power Use

16. Secrets: Protecting Your Keys and Private Data
17. Storage and Resource Tuning
18. First Steps as a Developer
19. Monitoring: Exporting Your Node's Metrics

Part VIII — Uninstalling

Appendices

Appendix A — CLI Quick Reference
Appendix B — Default Ports and File Locations
Appendix C — Exit Codes
Appendix D — Glossary
Appendix E — Manual Revision History & Growth Chart

About This Manual

This manual is a **living document**. It lives in the Freenet source repository (`docs/user-manual/freenet-user-manual.md`) and is updated alongside the software itself, so what you read here always describes a specific, named Freenet release (see the cover page). A print-ready PDF is regenerated from this file with the included `build-pdf.py` script.

How updates are highlighted

Every time the manual is revised, changes are marked so returning readers can see at a glance what is new — and so the manual doubles as a **visual growth chart** of both the software and your own journey with it:

- NEW marks a section added in the current revision.
- UPDATED marks a section whose content changed in the current revision.
- **Appendix E** contains the full revision history and the growth chart: which sections existed at each revision, and how coverage has expanded over time.

This is revision 1.4. Badges in this edition mark what changed between Freenet v0.2.135 and v0.2.136; the previous revision's badges have been retired, and every badge here traces to a row in Appendix E's delta ledger, which names the release that drove it (or marks it editorial). Appendix E also keeps the earlier revisions' ledgers, so you can see what each edition changed even after its badges come down.

How this manual is organized — the reader's growth path

The manual is arranged as a ladder. Start at the level that matches you today and climb as you advance:

Level	You want to…	Read
🟢 **Newcomer**	Understand Freenet and get it running	Parts I–II
🔵 **Everyday user**	Use apps, keep the node healthy	Parts II–IV
🟠 **Operator**	Configure, diagnose, fix, and upgrade	Parts III–VI
🔴 **Power user / developer**	Tune resources, manage secrets, monitor, build apps	Part VII

Conventions

- Commands you type are shown `like this` . Lines starting with `$` are typed at a terminal (don't type the `$`).
- `freenet` is the node binary; `fdev` is the separate developer tool.
- File paths are given for Linux first; Appendix B lists the macOS and Windows equivalents.
- Freenet is under active development. Where behavior is version-dependent, the manual says so and names the release.

Part I — Understanding Freenet

1. What Freenet Is

Freenet is a peer-to-peer network that turns the computers of its users into a single, resilient, distributed platform on which anyone can build and use decentralized services. There are no central servers to fail, censor, or de-platform you: every peer contributes storage, bandwidth, and computation to a fault-tolerant collective, and every application built on Freenet is interoperable by default.

From your point of view as a user, Freenet is deliberately boring:

1. You install a small program (the **Freenet node**, the `freenet` binary).
2. It runs quietly in the background as a system service.
3. You open Freenet applications **in your normal web browser**, served by your own node at a local address — no special browser, no plugins.

Under the hood, your node connects over UDP to other peers, forming a "small-world" network that can find and synchronize data quickly without any central index.

1.1 The pieces you'll encounter

Piece	What it is
Node (peer)	The `freenet` program running on your machine. Stores data, routes traffic, executes contracts.
Gateway	A publicly reachable node that helps new peers join the network. Your node fetches a list of gateways from `freenet.org` on first start.
Contract	A small WebAssembly program plus its associated public state, replicated across peers. Contracts define what state is valid and how it merges — this is how Freenet keeps data consistent without servers.
Delegate	A WebAssembly program that runs **only on your node**, managing your private data and keys — never replicated to other peers.
App / web app	A normal web application (HTML/JS/WASM) published on Freenet and loaded through your local node in your browser.
Dashboard	Your node's local web page at `http://127.0.0.1:7509/` showing status and available apps.
WebSocket API	The local API (port 7509) that apps and tools use to talk to your node.

1.2 What Freenet is not

- It is **not** a VPN or an anonymizing proxy for the ordinary web.
- It is **not** the same software as the 1999 "Freenet" (that project is now called Hyphanet); this is a ground-up redesign, sometimes called "Freenet 2023" or Locutus during development.
- It does **not** require you to be a developer. Running a node and using apps needs no programming at all.

1.3 Where to learn more

- Website: https://freenet.org/
- Whitepaper (architecture deep-dive): https://freenet.org/whitepaper/
- Community chat (Matrix): `#freenet-locutus:matrix.org` — this is the room the developers actually use. Older links point at `#freenet:matrix.org`, which is not the project's room; if you joined that one, switch.
- Source code: https://github.com/freenet/freenet-core

Part II — Getting Started

2. Installing Freenet

2.1 System requirements

- **OS:** Linux (x86_64/aarch64), macOS, or Windows.
- **Memory:** 1 GB+ available RAM recommended.
- **Disk:** by default Freenet sizes its hosted-contract-state budget from available memory — one eighth of it, never more than 1 GiB (configurable, and raising it is how you contribute disk — see §17) — plus logs and caches.
- **Network:** an ordinary internet connection. Freenet uses UDP (default port 31337) and works behind most home routers/NAT without configuration.

2.2 Recommended install (Linux and macOS)

Run the official installer:

```
curl -fsSL https://freenet.org/install.sh | sh
```

The installer:

- downloads the latest signed release binary into `~/.local/bin/`;
- sets up a **supervised service** (systemd on Linux, launchd on macOS) so the node starts automatically and — crucially — **auto-updates** (see Part VI);
- on Linux, prefers a system-wide service when it can elevate (root or sudo), otherwise installs a user service with *lingering* enabled so the node keeps running when you're logged out.

Two installer fixes landed in v0.2.135 that matter if the first install didn't take cleanly:

- **SELinux file contexts are restored after the binaries are copied.** On Fedora, RHEL and other SELinux systems, a binary installed into a new location could end up with a label that stopped systemd from executing it — a failure that looks like a broken install rather than a policy problem. If you hit that before, re-running the installer now fixes it.
- **A user-level binary now gets a user-level service.** Previously the choice of service scope could disagree with where the binary actually lived, producing a service that pointed at a path it wasn't allowed to run.

You can customize it with environment variables before the command:

Variable	Effect
`FREENET_INSTALL_DIR`	Install directory (default `~/.local/bin`)
`FREENET_VERSION`	Install a specific version instead of the latest
`FREENET_NO_SERVICE=1`	Install the binary only, **no** service. The node will not auto-update until you set up supervision.

> **Why the service matters:** a Freenet node never replaces its own binary while running. When it detects a new release it *exits with code 42* and relies on its supervisor to run `freenet update` and restart it. Without a supervisor, a node detects the update, exits… and stays stopped on the old version. Install the service unless you know you want manual control.

2.3 Desktop apps: macOS and Windows

macOS now has a first-class app: download the DMG from https://freenet.org/, drag Freenet to Applications, and launch it. The app runs the node under a menu-bar wrapper (the same supervision — auto-update, crash backoff, log capture — that systemd provides on Linux) and, on first launch, automatically puts the `freenet` and `fdev` command-line tools on your `PATH` (symlinked into `/usr/local/bin`), so the terminal commands in this manual work without any shell configuration. *(App since v0.2.125; automatic CLI setup since v0.2.133.)* The §2.2 installer script also still works on macOS if you prefer a service without the app.

Menu bar icon on macOS 26 (since v0.2.155). On macOS 26, earlier versions of the app could start the node without showing the Freenet icon in the menu bar. The node ran, but you had no menu to stop or restart it. Since v0.2.155 the app's main executable is the Freenet binary itself instead of a launcher script, so the icon appears. If you turned on Launch at Login with an older version, the first start of v0.2.155 or later updates that login entry for you, and you do not need to do anything. If the update cannot be written, the old entry is kept and keeps working. If the icon is still missing after updating, quit Freenet (for example from Activity Monitor) and open Freenet.app from Applications again.

Windows: download the Windows installer from https://freenet.org/. The node runs under a **tray application**: a Freenet icon in the system tray with Start/Stop/Dashboard controls, with the same supervision as above. On first launch it opens the dashboard in your browser. Since v0.2.131, Windows release binaries are **Authenticode-signed**, so SmartScreen warnings about an unknown publisher no longer apply to current releases.

2.4 Installing from source (advanced)

With a Rust toolchain installed:

```
git clone https://github.com/freenet/freenet-core.git
cd freenet-core
cargo install --path crates/core     # the node:  freenet
cargo install --path crates/fdev     # dev tool:  fdev (optional)
```

Then set up the service yourself: `freenet service install` (add `--system` for a root-owned, system-wide service). Note that **locally built ("dirty") dev builds disable auto-update** on purpose, so an update can never clobber your local changes.

2.5 Verifying the install

```
$ freenet --version
```

This prints the version plus the exact git commit and build timestamp — you'll use it again in the self-check (Part IV).

2.6 Docker

Since v0.2.133 there is an official container image, `ghcr.io/freenet/freenet-core`, published for every stable release (tags: exact version like `v0.2.133`, minor series `0.2`, and `latest`; built for `linux/amd64` and `linux/arm64`):

```
docker run -d --name freenet-node --network host \
    -v freenet-data:/data --restart unless-stopped \
    ghcr.io/freenet/freenet-core:latest
```

Then open `http://127.0.0.1:7509/` as usual. Three things to know:

- **The container self-updates.** The image's entrypoint plays the supervisor role (§2.2): it catches the node's exit-42 "update me" signal, applies the update, and restarts. You do not need Watchtower, cron, or manual `docker pull` to stay current — a container started once keeps itself up to date.
- The node runs from `/data/bin/freenet` on the **volume**, not from the image layer, so applied updates survive `docker compose down && up`. On start, the newer of (image binary, volume binary) wins — pulling a newer image never rolls a self-updated node backwards.
- `docker exec freenet-node freenet --version` reports the version actually running, which may be ahead of what the image shipped with.

A ready-made `docker-compose.yml` lives in `docker/freenet-node/` in the source repository, alongside the full container documentation.

2.7 Nix

New in v0.2.136. Nix is now a **supported deployment path**, not merely a way to get a compiler:

```
nix run github:freenet/freenet-core          # the supervised, self-updating node
nix build github:freenet/freenet-core#freenet  # build only: the bare binary
nix develop                                   # dev shell: pinned toolchain, nextest, …
```

The distinction between the two outputs is the whole point, and it is easy to get wrong:

Output	Auto-updates	Use it for
`packages.freenet-node` (also `packages.default`)	**Yes**	Running a peer. The only supported way to do that.
`packages.freenet`	**No**	Building and development only — CI, `nix develop`, packaging.

`packages.freenet` is *just the binary*: nothing seeds it into a writable location and nothing restarts it, so a peer started from it never updates itself and will fall behind the network's compatibility floor (§13).

> **The overlay trap.** If you apply `overlays.default`, `pkgs.freenet` is the **bare** binary. Writing `environment.systemPackages = [ pkgs.freenet ];` puts a `freenet` on every user's PATH that carries no supervisor, never updates itself, and says nothing about either. Use `pkgs.freenet-node` for anything that runs a peer.

Why the self-updating node doesn't run from `/nix/store`. `freenet update` replaces the *running* binary in place, and the Nix store is read-only — so the updater cannot run from a store path. Rather than make Nix the updater, `freenet-node` seeds the Nix-built binary once into a mutable state directory (`$STATE_DIRECTORY/bin/freenet` under systemd, otherwise `${XDG_STATE_HOME:-$HOME/.local/state}/freenet/bin/freenet`) and supervises it from there. Arguments are forwarded, so `nix run … -- --config-dir /srv/freenet` works as you'd expect. The older name `freenet-autoupdate` still works as an alias.

On NixOS, use the module (v0.2.140). The flake now exports `nixosModules.default`, which runs the supervised, self-updating peer as a system service for you:

```
modules = [
  ./configuration.nix
  freenet.nixosModules.default
  { services.freenet-node.enable = true; }
];
```

Option	Default	Meaning
`services.freenet-node.enable`	`false`	Run the peer as `freenet-node.service`.
`services.freenet-node.package`	the flake's `freenet-node`	Must provide `bin/freenet-node`; the bare `freenet` package has no supervisor.
`services.freenet-node.extraArgs`	`[ ]`	Passed through to `freenet network`.

State lives in `/var/lib/freenet` (mode 0700, because the config directory holds the node's keys), which is also the `freenet` user's home.

> **The name trap.** It is `services.freenet-node`, **not** `services.freenet`. In nixpkgs, `services.freenet` is an alias for `services.hyphanet` — the unrelated Java project once called Freenet. Enabling the wrong one gives you a working service for a different network, with nothing to tell you so.

If you write the systemd unit yourself, keep the service user's `home`. The node keeps its auto-update state — the crash-probation marker, the known-good rollback snapshot, the known-bad version pin — under the service user's home. A NixOS user declared without `home` gets `/var/empty`, where every write fails and **crash-loop rollback (§13) is silently off**. Since v0.2.140 the node falls back to `$STATE_DIRECTORY` when the home is unusable, but only systemd sets that variable — so a manual `freenet update` run from a shell must be given it explicitly, or it will neither see the node's known-bad pin nor clear its failure counter:

```
sudo -u freenet STATE_DIRECTORY=/var/lib/freenet /var/lib/freenet/bin/freenet update
sudo systemctl restart freenet-node   # a manual update replaces the binary, not the runni
```

Seeding from a release older than v0.2.140 runs a binary without that fallback first, so setting `home` is still worth doing.

Full details are in `docs/nix.md` in the source repository.

3. First Run and the Dashboard UPDATED

If you installed with the service (the default), the node is already running. Open the **dashboard**:

> **http://127.0.0.1:7509/**

The dashboard is served by your own node, on your own machine — it works even though it's "a website" because your node includes a small local web server. From here you can see node status and launch Freenet applications. The dashboard has grown considerably since v0.2.123: it now shows a **measured GET success rate** (real fetch outcomes, not a synthetic health verdict), offers a **per-contract detail page** at `/contract/{key}` for anything your node hosts, follows your OS light/dark theme, and its tables filter and collapse for small screens.

Peer pages, and the new `/routing` page (v0.2.142). Each peer your node is connected to has its own page (`/peer/{address}`, since v0.2.136). In v0.2.142 it was redesigned around one question: *what has your node learned about this peer?* It compares the peer with what distance alone would predict — how much slower or faster it replies, how often its requests fail — and with your node's other peers, and it shows how often the peer was a candidate in a routing decision and how often it was actually chosen. Until a peer has handled enough requests, the page says so plainly (*"Still learning about this peer"*) instead of drawing a confident chart from a handful of data points. Since v0.2.143 the same plain speech covers the opposite case: when your node has enough data but sees **no measurable difference** between its peers, the page says exactly that — *"No measurable difference between your peers' response times yet"* (and likewise for failure rates) — rather than drawing a chart that implies a ranking the node hasn't learned. That is normal on a young or quiet node, not a fault.

The network-wide picture moved to a new page, `/routing`: how much the router has learned overall and how good its predictions are across all peers. That is now where the **skill score** lives — how much better the router's estimates are than assuming nothing, with a negative score still spelled out in words rather than shown as a bare `-0.43`. *Earlier editions described that score as the headline of each peer page; since v0.2.142 it isn't.*

Since v0.2.138 the dashboard also carries an **Apps and permissions** card, linking to `/permission/apps` — the page listing every permission you have granted an app, with a button to revoke any of them. §4.1 explains what those permissions are and when you will be asked for one. (That page had no way back to the dashboard when it shipped; v0.2.139 adds the link.)

A live bandwidth chart (since v0.2.145). The **Data Transfer** card now has a section called *"Transfer rate, last minute"*. It charts your node's upload and download rates over the last 60 seconds, measured once a second, and shows the latest figures above the chart (*"Latest: Upload … Download …"*). This lets you see what your node is doing right now, where before you only had lifetime totals. Use it to check whether a slow app or a busy connection is Freenet's doing. On a desktop-width screen, hover over the chart to read any single second. Just after the node starts, the section reads *"Measuring transfer rates…"* until it has two readings. If the newest reading is more than 3 seconds old, the figures carry *"as of N s ago"* so you don't mistake a stalled reading for the current rate. The chart counts Freenet traffic only, as encrypted UDP datagrams without UDP or IP headers, so it will read a little lower than your router or OS counters. The rates use decimal units (1 KB/s = 1,000 bytes per second), while the card's lifetime totals above them use binary units. The measurement stays on your machine: it sends nothing to the project's telemetry, and there is nothing to configure.

Red no longer means "normal" (v0.2.141). Until this release the dashboard painted some perfectly ordinary states in warning colours: the operations failure count was always red, *even when it read 0*, and a contract marked "next to evict" was orange on a node far below its budget. Both are now neutral. A full hosting cache is labelled *"Full is normal here"* rather than coloured, because a cache is supposed to fill. The practical rule: if you learned to ignore red on the dashboard, unlearn it. Red now means something.

If you skipped the service, you can run a node in the foreground:

```
$ freenet network
```

Leave it running in that terminal and open the dashboard as above. (`freenet` with no subcommand does the same thing — network mode is the default.)

The first startup fetches the current **gateway list** from `https://freenet.org/keys/gateways.toml` (if that fetch fails, it uses its cached copy plus the gateways built into the binary; see §11.4), connects to a gateway over UDP, and joins the network. Within a short time the node acquires peer connections of its own and no longer depends on the gateway.

4. Using Freenet Applications

Freenet apps are ordinary web apps loaded through your node. The flagship application today is **River** (https://freenet.org/) — decentralized group chat with invitation-based rooms, no server, no account with any company.

General pattern for any Freenet app:

1. Make sure your node is running (dashboard loads).
2. Open the app's local URL in your browser (linked from the dashboard or from the app's own site).
3. The app talks to your node over the local WebSocket API; your node fetches and synchronizes the app's contracts with the network.

Because apps run against *your* node, they keep working as long as your node does — there is no "server down" failure mode, only your own machine.

What an app tells you when something goes wrong (v0.2.134). Three long-standing cases where the node reported the wrong thing to the app were fixed in this release. They need no action from you; they matter because an app that looked broken may simply have been mis-reported:

- A large upload sent in chunks (a "streaming PUT") could be reported as failed when it had in fact been stored — the internal watchdog outlived the success reply. Such uploads now report success.
- A contract already stored on your node could be reported as unavailable when an unrelated network fetch failed transiently. Local state is no longer suppressed by a passing network error.
- Delegate failures (the programs that hold your private keys and data) are now propagated to the app instead of being swallowed, and delegates that subscribe to a contract are notified when its state is first installed rather than only on later changes. An app that seemed to hang on first use should now either work or tell you why.

Apps can go fullscreen (v0.2.136). The sandbox your node runs an app in now permits fullscreen, so a Freenet app can fill the screen like any ordinary web app. A second fix in the same release means a **streaming upload that fails while being relayed now reports the failure upstream instead of going quiet** — the same class of "looked like a hang, was actually an unreported error" that §4's earlier fixes addressed.

Delegates can reach the network now (v0.2.135). Previously a delegate — the part of an app that runs only on your machine and holds your private data — could only see contract state your node already had. Delegates can now perform GET and SUBSCRIBE over the network, so an app's private half can fetch and follow contracts directly instead of routing everything through its web front-end. Expect apps to feel less dependent on having a browser tab open.

Two limits worth knowing, because an app may bump into them rather than fail outright. A single delegate may subscribe to at most **256 contracts**, and when it reaches that ceiling the node **evicts its least-recently-notified subscription** rather than refusing the new one — so a busy subscription stays live while a dormant one is quietly dropped. The cap is deliberately **not configurable**: a tunable limit would mean the same app works on some peers and not others, which is precisely the non-uniformity Freenet avoids.

Delegate subscriptions now survive a restart (v0.2.137). Until this release, restarting your node silently dropped every subscription a delegate held: the private half of an app stopped being notified of contract changes until something re-subscribed it. Those subscriptions are now written to disk and replayed at startup, so notifications come back on their own.

Three details are worth knowing, because "on their own" is not the same as instantly:

- **There is a warm-up.** The node waits roughly **15 seconds** after starting before it re-establishes the first subscription — long enough to have joined the ring — then works through them one at a time. A delegate that looks quiet in the first few seconds after a restart is not broken.
- **Two ceilings.** At most **4096** subscriptions are kept on disk, and at most **512** are re-established *over the network* per boot. Anything past 512 is still restored locally, so a contract your own node updates still notifies the delegate; it simply doesn't get its network subscription back until a later boot or until the app re-subscribes.
- **Upgrading a delegate starts it over.** A new version of a delegate is a new key, and subscriptions deliberately do not migrate to it. After an app updates its delegate, expect that delegate to re-subscribe from scratch.

A stranded-transfer bug is fixed (v0.2.138). When a large transfer's data fragments overtook the message describing them — ordinary network reordering — the two halves could fail to find each other and the transfer was left hanging. Both orderings are now handled, so a large upload or download no longer strands itself on a reordered network. This is the same family as the misreporting fixes above: nothing you did wrong, and nothing you could have seen from the app.

Apps left open no longer go quiet after half an hour (v0.2.147). Many apps get live updates through their delegate, the private part of the app that runs on your node. New messages in a chat are one example. Before v0.2.147, your node stopped sending those updates to an open tab 30 minutes after the app last sent a request to its delegate. An app that set itself up once and then only listened would stop showing new activity, even though the tab was still open and connected. Updates came back for another 30 minutes when you reloaded the tab, or when you did something in the app that made it contact its delegate again. Since v0.2.147, your node keeps sending updates to a tab for as long as it stays connected. It stops only when the tab is closed, its connection drops, or the delegate is removed. There is nothing to configure, and you no longer need to reload an idle app to keep it current.

If you run a node that many people reach through a reverse proxy: a single delegate can send live updates to at most 128 connected app tabs at a time. A tab now keeps its place for as long as it stays connected, not just for 30 minutes. A tab that connects when all 128 places are taken still loads and still gets answers to its own requests, but it gets no live updates until another tab disconnects.

Saves no longer pause for minutes (since v0.2.147). When an app saves something, such as a message, a post or a settings change, your node first sends a short summary to a nearby peer to see whether it already has that data. If the reply to that check got lost, older nodes waited a timeout that grew with the size of the app's data before they sent the full data. For River rooms that wait was about 1.5 to 2.5 minutes. The save did go through in the end, but the app looked frozen in the meantime. Since v0.2.147 the node stops waiting for a lost reply after a few seconds (at most about 15 with default settings) and sends the full data straight away. You don't need to do anything, and there is nothing to configure. If your node log shows the line "PUT summary-first: probe timed out; falling through to full-state driver", this fallback ran. It is normal and the save still completes. If a save still takes minutes, the cause is something else: work through §7 and §11.

4.1 When an app asks to run in the background UPDATED

Normally the private half of an app — its delegate — runs only while the app is open. From **v0.2.138** an app can ask to keep running with **no tab open**: a shop that answers a buyer while the seller is away, for instance. This is the first thing your node asks your permission for, so it is worth understanding before you meet the prompt.

The prompt comes from Freenet, not from the app. The node writes it and the node shows it. An app cannot draw its own version, phrase it more persuasively, or answer it on your behalf. If you are being asked whether something may run in the background, that question came from your own node.

You are asked once, and the answer belongs to the *app*, not to the delegate. That distinction is what makes "once" mean once:

- Choosing **Allow** stores the grant. The app shipping a new version of its delegate — a routine event — does **not** ask you again.
- What *does* ask again is the app itself being re-keyed, and that also changes the app's URL, so a repeat prompt is a visible signal rather than a silent one.
- Choosing **Not now** is remembered too: the app cannot ask again for **seven days**. No app can wear you down by asking repeatedly.
- Dismissing the prompt without answering stores nothing, so you will be asked next time. Ignoring is not consent — and it is not refusal either.

Reviewing and revoking. `/permission/apps`, linked from the dashboard's *Apps and permissions* card, lists every grant and revokes any of them. Nothing is permanent because you clicked once.

From v0.2.139 a granted app can also run on a schedule. Until this release the permission meant two specific moments: once when the app was installed, and once each time your node started. A delegate can now also declare *periodic wake-ups* — "run me every N seconds" — and the node fires them.

The reason this does not need a second question from your node is worth being precise about, because it is the whole safety argument:

- It is the **same grant**, re-checked at **every** fire. Revoking on `/permission/apps` stops the scheduled runs too, immediately and without a separate control to find.
- It is the **same budget** described below. Scheduled runs are not an extra allowance on top of the background one; they draw from it.
- The schedule is **bounded by the node, not by the app**: no faster than once a minute, no slower than weekly, and at most **four** schedules per delegate. Only one run per schedule can be pending at a time, so a delegate cannot pile up missed fires and then stampede.
- If your node is busy, or the delegate has spent its budget, a fire is **deferred rather than run** — and a fire deferred repeatedly is dropped instead of queued forever.

Nothing about a pending fire survives a restart; schedules are simply re-armed when the node starts. So a wake-up missed while your machine was off does not arrive in a burst when you turn it back on.

Background work is budgeted. A delegate running unprompted is limited in how long it may run and how many contract operations it may perform, per delegate and across the node as a whole. One that exhausts its budget is refused with a "retry later" rather than being left to spin — so granting this permission cannot hand one app your whole node.

> **What this permission does not protect you from.** It governs *apps*. It is not a barrier against other software already running on your computer: any local program that can open a connection to your node can drive a delegate regardless of these grants, and could do so before this feature existed. Read the permissions page as a record of what you have allowed *apps* to do, not as a defense against your own machine. Protecting the machine itself remains the machine's own job.

Only your own machine can remove a delegate (since v0.2.150). A delegate can now be unregistered only by an app or tool connected to your node from the same machine; a request from anywhere else is refused with "only a local client may unregister a delegate". Unregistering never deletes the delegate's stored secrets.

Apps can no longer pretend to be your node when talking to a delegate (since v0.2.155). An app, or any program connected to your node, may send a delegate only its own app messages. Messages that only your node should send are now refused before the delegate runs. These include contract update notifications, contract read or write results, your answers to the delegate's prompts, messages from other delegates, and start-up or wake-up signals. Message kinds the node does not recognise are refused too. Before v0.2.155 only start-up and wake-up signals were refused. An app could hand a delegate made-up contract state, a made-up answer from you or a made-up message from another delegate, and the delegate could not tell it from the real thing. A refused request returns "a client may send a delegate only ApplicationMessage; is delivered by the node", where names the refused message, and the node logs a warning: "Refused a client delegate request carrying an inbound message only the node delivers". There is nothing to configure. If an app starts showing this error after you upgrade, the app is relying on behaviour that is no longer allowed. Report it to the app's author.

Desktop programs can ask to use an app (since v0.2.156). A program running on your own computer, such as a desktop client, can ask your node to let it talk to delegates as one particular Freenet app, with that app's data. The program opens a page from your node in your browser, titled "Allow a program to use a Freenet app?". The page shows the app's contract id and the name the program gave itself, and marks that name as unverified. The Allow button becomes clickable after a second and a half. Only allow it if you just started that program and asked it to connect; otherwise choose Deny. A request you do not answer expires after 10 minutes.

If you allow it, the program keeps a token and can reconnect after restarts without asking again. Every such grant is listed under "Programs using an app" on `/permission/apps`, where Revoke cuts the program off at once and closes any open connection. Your node holds at most 64 of these grants; once it is full, new requests are refused with a message asking you to revoke one on the Apps and permissions page.

These requests are accepted only from the same machine. Changing `--ws-api-address`, `allowed-host` or `allowed-source-cidrs` does not let another computer ask, and requests through a reverse proxy that adds forwarding headers are refused. A node in hosted mode does not offer them. As with background permission, approval does not protect you from malicious software already running as your user, which could open and answer the page itself.

If you run a node for other people with `--hosted-mode` (env `FREENET_HOSTED_MODE`, config `hosted-mode`, off by default), since v0.2.150:

- The node refuses any client-API connection without a user token (`userToken`), answering HTTP 403 "this hosted Freenet node requires a user token (userToken) on the client API". Older clients that left the token out used to land in the node's shared space and now cannot connect.

- Each delegate runs as the user who asked for it; a delegate it calls also runs as that user and sees only that user's secrets.
- The node never runs a delegate on its own: no contract-change notifications, start-up runs or periodic wake-ups. A delegate can still subscribe to a contract, which keeps it up to date for the user's own reads.
- Unregistering removes only that user's registrations. To remove the delegate itself, restart without hosted mode and unregister it locally.
- Imports into the node-local (shared) secret scope are refused.
- Permission prompts require a user token, and each user sees only their own.

What visitors to a hosted node see (since v0.2.156). Every app on a hosted node has a bar above it that the app cannot hide. It now reads "Stored encrypted on this server. Install Freenet for full privacy." Before v0.2.156 it read "Hosted proxy, not private. Your data is stored on this server." and gave no way forward. "Freenet" in the bar links to https://freenet.org/, and "Install Freenet" links to https://freenet.org/quickstart/. The Account menu's "Your own peer" section and the hint shown after "Move to my peer" also link to the quickstart, so a visitor with no peer of their own knows where to get one. These links open in a new tab and leave the hosted session where it was. There is nothing to configure, and nodes without hosted mode show no bar.

A node that does not use hosted mode behaves as before, apart from the local-only unregister rule.

4.2 Opening `freenet:` links

From **v0.2.140**, a Freenet app can be shared as a link. The *Open in Freenet* button on https://freenet.org/open produces one of the form `freenet:<contract-id>…`, and clicking it opens that app **in your browser, through your own node** — the same `http://127.0.0.1:7509/…` address you would reach from the dashboard.

It is set up for you. `freenet service install` registers the link handler (so do the install script, the Windows setup wizard and `freenet service doctor`). An install that predates this release is caught up the first time the updated node starts — but only where the node is launched the standard way (the Linux *user* service, or the Windows startup entry); for anything else, run `freenet service url-handler register` once. How it is registered depends on the platform:

Platform	Registration
Linux	A desktop entry in `~/.local/share/applications/`. It becomes the default handler **only if no other handler is already set** — a choice you made is never replaced.
Windows	A per-user registry entry (`HKCU`), so no administrator rights are needed.
macOS	Declared by the `Freenet.app` bundle itself, which is launched if it isn't running. A bare binary outside the app bundle cannot receive links on macOS.

To opt out, run `freenet service url-handler unregister`. That removes the registration **and** stops the node re-adding it on start. `freenet service url-handler register` puts it back. Installs without the service (Nix, `cargo install`, a hand-run binary) use `register` to opt in.

What a link can and cannot do. Any website can fire a `freenet:` link at your computer, so the handler treats every link as hostile:

- A link can only choose **which contract** to open. The scheme, host and port are fixed by the handler — always your own node at `127.0.0.1` — so a link cannot send your browser anywhere else.
- The link is passed to the browser as a single argument after a literal `--`, never through a shell, so it cannot smuggle in extra options.
- The handler **never starts a stopped node** (that would fight with the service supervisor). If your node isn't running you get a local page saying so, rather than a silent failure. On macOS, opening the link launches `Freenet.app` exactly as a double-click would.

Links that were shared earlier in the `freenet://…` form still work. `freenet.org/open` now emits the form without `//`, because some desktops (KDE among them) lowercase what they take to be a host name, and in the `//` form that is the contract id, which is case-sensitive.

If your node is still connecting. Right after the node starts, it needs a moment to join the network. If you open an app link during that time, you see a *Connecting to Freenet...* page. Since **v0.2.145**, that page stays on the app's address and tries again every few seconds for up to two minutes. The app opens by itself once the node has joined, with the link exactly as it was shared, including any parameters and `#…` part. Earlier versions sent you to the dashboard after three seconds, so you lost the link and had to open it again. If the node still hasn't connected after two minutes, you are taken to the dashboard, where you can check its status. While the page waits, your address bar may briefly show an extra `_freload=…` parameter. Freenet uses it to limit how long the retries run. It is removed once the app loads, and the app never sees it. (Retrying needs JavaScript. With JavaScript turned off, you still go to the dashboard.)

Live updates right after startup. An app you open while your node is still joining the network follows its contracts through the gateway your node first connected to. That gateway can answer your node, but it can't pass on later changes to it. Before **v0.2.146**, such an app could load normally and then show no new messages or changes for several minutes (up to about six), until the node's regular renewal happened. Since **v0.2.146**, your node remembers which subscriptions it made this way. As soon as it has a proper connection to the network, it renews them through that connection. If that doesn't work, it tries again every two minutes until it does. There is nothing to configure. If an app opened right after a restart seems frozen, wait a minute or two before you reload it.

Part III — Operating Your Node

5. The Service: Day-to-Day Control

All service management goes through `freenet service …`. On Linux these commands drive systemd (user service by default; add `--system` everywhere if you installed system-wide); on macOS, launchd; on Windows, the tray wrapper.

Command	What it does
`freenet service status`	Health summary: is the service installed, running, which binary/version, key paths. **Start here whenever something feels off.**
`freenet service start` / `stop` / `restart`	Start, stop, or bounce the node.
`freenet service logs`	Follow the node's log output live.
`freenet service logs --err`	Only error-level logs.
`freenet service disable`	Keep the node stopped **persistently** — it will not come back on reboot or re-login until you re-enable. (The supervisor stays installed; only the node idles.)
`freenet service enable`	Undo `disable` and start the node again immediately.
`freenet service doctor`	Repair a wedged install (see §11.3).
`freenet service report`	Generate/upload a diagnostic report (see §10).
`freenet service install` / `uninstall`	Add or remove the service itself.

Stopping vs disabling: `stop` is temporary — the service starts again on the next boot. `disable` writes a persistent marker that makes the node refuse to run until you `enable` it again. Use `disable` when you want Freenet off for a while on a laptop, for example.

> **Stopping the node on purpose is no longer mistaken for a crash** (v0.2.135). The supervisor counts crashes to decide whether a freshly updated version should be rolled back (§13). A deliberate `freenet service stop` — or a `systemctl stop` — used to be counted the same as a crash, so stopping your node a few times shortly after an update could trip the crash-loop detector and revert a version that was working perfectly. Deliberate stops are now recognized as deliberate.
>
> **A restart no longer costs you delegate subscriptions** (v0.2.137). A bounce used to leave every delegate — the part of an app that holds your private data — unsubscribed from the contracts it had been following, which is why some apps needed reopening after a restart. Those subscriptions are now restored from disk; §4 covers the brief warm-up and the two ceilings.

6. Configuration

6.1 Where things live (Linux)

What	Default location
Config directory	`~/.config/freenet/`
Data directory (contract state, databases, caches)	`~/.local/share/freenet/`
Logs	under the data/log dir shown by `freenet service status`
Gateway cache	`gateways.toml` in the config directory
Secrets (encrypted delegate keys)	`secrets/` under the data directory

Override with `--config-dir`, `--data-dir`, `--log-dir` flags or the `CONFIG_DIR`, `DATA_DIR`, `LOG_DIR` environment variables. See Appendix B for macOS/Windows paths.

> **Your config file is `config.toml` — exactly that name** (v0.2.135). The node used to scan the config directory and could pick up a near-miss such as `config.bak.toml` or `config.old.toml` instead of the real file, silently running on settings you thought you had retired. It now loads `config.toml` directly when it exists and does not scan the directory at all. Keeping backup copies alongside your live config is safe again — but if you have been relying on a differently named file being found, rename it to `config.toml`.

6.2 Options you're most likely to touch UPDATED

Every option can be given as a CLI flag or an environment variable:

Option	Default	Meaning
`--ws-api-port`	`7509`	Local WebSocket API + dashboard port.
`--network-port`	`31337`	UDP port for peer-to-peer traffic.
`--address`	`::` (dual-stack)	Bind address for the network listener.
`--log-level` (`LOG_LEVEL`)	`info`	`error`, `warn`, `info`, `debug`, `trace`.
`--max-hosting-storage`	**RAM ÷ 8**, clamped 128 MiB–1 GiB	Budget for hosted contract **disk** state; least-valuable contracts are evicted beyond this (§17). Not a flat 1 GiB — 1 GiB is the *cap*.
`--hosting-mem-share` (`HOSTING_MEM_SHARE`)	`0.125` (⅛)	Share of the node's **memory limit** that hosted contracts may hold in **RAM** — the RAM counterpart of `--max-hosting-storage` (§17). Never below 64 MiB; above `0.5` draws a startup warning.
`--max-blocking-threads`	2×CPU (4–32)	Worker threads for WASM execution.
`--telemetry-enabled` (`FREENET_TELEMETRY_ENABLED`)	on during alpha	Sends operation timing and network topology to the project's dashboard. Contract content is never included.
`--telemetry-endpoint` (`FREENET_TELEMETRY_ENDPOINT`)	`http://telemetry.freenet.org:4318`	Where that telemetry goes (see the note below).
`--otel-telemetry-enabled` (`FREENET_OTEL_TELEMETRY_ENABLED`)	`false`	Export **your own** node's metrics to **your own** OpenTelemetry collector (§19). Entirely separate from `--telemetry-enabled`.
`FREENET_ROUTING_HIERARCHICAL`	*(obsolete)*	**No effect since v0.2.142.** The hierarchical estimator always routes, and the older routing stack this switch used to select has been

Option	Default	Meaning
		removed, so there is no longer a way back to it. A node that still sets the variable logs a warning saying so. Remove it from your environment or unit file.

> **Don't reach for `FREENET_ROUTING_FALLBACK_ISOTONIC` as a replacement.** It sounds like the old switch's successor, but it is an *emergency* fallback that routes on a configuration no release has ever shipped as a whole — not the old router. The node warns at startup while it is on. Leave it unset unless a developer asks you to set it for a specific problem.
>
> **The project telemetry endpoint moved in v0.2.134**, from `nova.locut.us:4318` to `telemetry.freenet.org:4318` — a role-based name, so the address survives future host moves. You do not need to do anything: the node stores the resolved endpoint in `config.toml`, and on the first start after upgrading it recognizes the old default and re-derives the new one. An endpoint you set yourself, by flag, environment variable, or by hand in the file, is left alone.

Capping your node's upload (since v0.2.148). If you set `total-bandwidth-limit` (in bytes per second, in `config.toml` or as `--total-bandwidth-limit`), it is now a true limit on everything your node sends: app data, handshakes, acknowledgements, keepalives and resends, across all connections together. Before v0.2.148 it only set the speed for each connection, so the node as a whole could send more than you asked for, especially with many peers or a high `min-bandwidth-per-connection`. Now `min-bandwidth-per-connection` is a floor inside the total. It never takes you over the limit, and if you set it higher than the total it is lowered to match. The limit holds on average, so short bursts can go over it. The **Data Transfer** chart's upload rate should settle at or below your limit. A value of 0 means no limit, the same as leaving it unset (since v0.2.156): there is no node-wide cap and no per-connection share, and `bandwidth-limit` applies if you set it. The log says `total-bandwidth-limit is 0, which is no limit: no node-wide egress cap and no per-connection share is applied, and bandwidth-limit applies if set`. Before v0.2.156, a total of 0 still held each UDP connection to about 1 MB/s and made the node ignore `bandwidth-limit`. When the limit is on, the log says so at startup (`Node-wide egress cap active ...`). It also warns you if the limit is too low for the number of connections your node allows.

Transfers on a busy connection take turns (since v0.2.149). Under `total-bandwidth-limit`, each connection now sends its transfers' pieces in turn instead of in random order, so the transfers on a connection move forward at the same pace; before, an unlucky transfer could wait more than 5 seconds for its next piece and fail partway through. With N transfers on a connection that gets C bytes per second, each sends a 1,200-byte piece about every N × 1200 / C seconds, so about 5 × C / 1200 transfers fit at once (roughly 80 at 20 KB/s). If a transfer's next piece would have to wait past the 5-second limit, the node refuses it at once instead of letting it stall: the transfer fails with "next fragment refused: its connection's queue would reach the egress deadline", and its operation retries as it normally does. If this happens often, raise `min-bandwidth-per-connection` or `total-bandwidth-limit`. Since v0.2.156 the node-wide limit is shared in turn too. Before, transfers waiting for it grabbed room in random order, the experimental QUIC path won most of it, and an unlucky transfer could wait past the 5-second limit and

fail. Now every waiting transfer and QUIC connection queues in one line and is served roughly in arrival order, with the same N × 1200 / C spacing as above, counting every waiting sender on the node. Connections are not isolated from each other: a peer that runs many transfers through your node gets a correspondingly larger share of the limit.

If you leave `total-bandwidth-limit` unset (the default) or set it to 0, none of the above applies to you.

Large updates on a slow upload are no longer sent twice (since v0.2.156). When your node sends a large contract state to a subscriber, it waits for the transfer to finish before marking the subscriber as up to date. Before v0.2.156 it waited only 2 minutes. On a slow or lossy upload, or under a low `total-bandwidth-limit`, a large state could take longer (a 31 MiB state needs about 270 KB/s to finish in 2 minutes), so the node treated the send as failed and sent the whole state again on the next update. Now the wait grows with the size of the transfer: at least about 4.5 minutes, and about 15 minutes for a 31 MiB state, just under how long the receiving peer keeps a slow incoming transfer open. The limit on how many large transfers your node starts at once is unchanged. There is nothing to configure.

Transfers stall or fail after setting a limit. A limit that is too low for the number of transfers running at once now makes those transfers slow down and can make some of them fail, where older versions would just have sent more than the limit. The log names the cause, at most once a minute. *"too many concurrent streams for the node-wide cap: raise total-bandwidth-limit"* means the total is too small. *"... raise min-bandwidth-per-connection or total-bandwidth-limit"* means a single connection's share is too small. Raise the setting it names, or remove the limit.

Per-connection speed (`bandwidth-limit`, works since v0.2.152). With no bandwidth options set, each connection to a peer is paced at about 1.25 MB/s (1,250,000 bytes per second), the rate of the default `fixedrate` congestion controller, and there is no node-wide limit, so with many peers the node can send that rate times the number of connections. *(Older help text and docs gave this default as 3 MB/s or 10 MB/s; those figures were wrong, and the behaviour has not changed.)* `bandwidth-limit` (bytes per second, in `config.toml` or as `--bandwidth-limit`) caps each connection separately. Before v0.2.152 it only set a connection's starting speed and was replaced almost at once, so it had no real effect; now it holds for the life of the connection. It can only slow a connection down, never speed it up, so 10 peers can still receive up to 10 times the figure. Short messages, acknowledgements and resends are not limited by it. When `total-bandwidth-limit` is set to a value above 0, `bandwidth-limit` is ignored (the log says `bandwidth-limit is ignored because total-bandwidth-limit is set`); use the total to limit what your node sends as a whole. A value of 0 is ignored, with the warning `bandwidth-limit = 0 is ignored; set total-bandwidth-limit to limit what the node sends`. A value below 65,536 (64 KiB/s) is raised to 65,536 with a warning beginning `bandwidth-limit is too low for long transfers to complete; raised to the minimum`. If you set a low `bandwidth-limit` in the past, large transfers will be slower after upgrading and very large ones, such as multi-megabyte contract states, can time out; raise or remove the value if that happens.

QUIC obeys your bandwidth settings (since v0.2.156). If your node runs QUIC (`shadow` or `prefer`, below), each QUIC path is now limited to the same speed as its UDP connection: `bandwidth-limit`, or the connection's share of `total-bandwidth-limit`, and data sent over QUIC counts against the node-wide total. Before v0.2.156 QUIC paths ignored both settings and could send many times the limit to nearby peers. There is nothing new to configure. Two caveats: once data is sent over QUIC, a peer whose traffic is split between UDP and QUIC can get up to twice its per-connection share, though `total-bandwidth-limit` still caps the node as a whole; and control messages and one-packet messages are not held to the per-connection limit, the same as on UDP. On a default node QUIC carries only control messages, so this mostly affects those.

Experimental QUIC switches (since v0.2.149, extended in v0.2.152, v0.2.155 and v0.2.156). Unset now means `shadow` on Linux and macOS. `transport-quic` (`--transport-quic`, `FREENET_TRANSPORT_QUIC`) takes `off`, `shadow` or `prefer`. Since v0.2.155, leaving it unset means `shadow` on Linux and macOS and `off` on Windows and other platforms. With `shadow` or `prefer` the node opens a QUIC endpoint on its existing peer port (31337/UDP by default, so no firewall change) and upgrades connections to peers that also run QUIC to a QUIC path. With `shadow` that path carries control messages only and all application data goes over the normal UDP transport. If the endpoint cannot be created, the node logs a warning and runs with QUIC off. The unset default is worked out each time the config is read and is never written to `config.toml`, so a node that never set the key follows the release it runs: a later release can turn QUIC off again without you doing anything. A value you pass once with `--transport-quic` or `FREENET_TRANSPORT_QUIC` is saved to `config.toml` and stays until you remove it there. To opt out, set `transport-quic = "off"`. An unknown or misspelled value is treated as `off` and logs "Unknown transport-quic value; QUIC stays off". Since v0.2.156, `prefer` can carry data: a pair of peers sends its traffic on the QUIC path only when both nodes run `prefer` and both offer data, and falls back to UDP if the path cannot be used. Each node makes its offer with `quic-data-percent` (`FREENET_QUIC_DATA_PERCENT`, 0–100). Without it the node uses the release's share, which is still 0 in v0.2.156, so a node with `prefer` but no `quic-data-percent` still sends no data on QUIC; `quic-data-percent = 100` with `prefer` offers data to every peer. With `prefer` and no `quic-data-percent`, your node also follows any later release that raises the share, without asking again. Since v0.2.156, `quic-max-paths` (`--quic-max-paths`, `FREENET_QUIC_MAX_PATHS`) is enforced: it caps how many QUIC paths the node holds at once, counting paths being offered or dialed. It defaults to `max-number-of-connections`, a larger value is lowered to that, `0` turns QUIC upgrades off, and a value that is not a whole number is ignored with a warning. On Linux and macOS a crash breaker guards QUIC, and because `shadow` is now the default there, it is armed on a stock node: once the QUIC endpoint is up, the node records it in `quic-crash-breaker.json` in its data directory, and if that run ends abnormally (a crash, an out-of-memory kill, a SIGKILL — including one systemd, launchd, Kubernetes or `docker stop` sends after its stop timeout — or closing the terminal of a foreground node), the next start keeps QUIC off for 24 hours and logs "The previous run with QUIC enabled ended abnormally, so QUIC stays off until the cooldown ends. Set `quic-ignore-crash-breaker = true` to run QUIC anyway". On a default node a trip still costs nothing, because by default no application data rides on QUIC. On a node that runs `prefer` with a non-zero `quic-data-percent`, a trip moves that node's data back to UDP for 24 hours. Clean stops, auto-update restarts, reboots and the macOS tray app's own Restart/Stop/Quit/update do not trip it; an unreadable marker file keeps QUIC off for that run only.

On Windows, since v0.2.155, the breaker works a different way: it counts short runs, using the same marker file. A run with QUIC enabled that ends in less than 10 minutes counts as short, whatever ended it (a fatal listener or storage error counts too). After three short runs in a row, the next start keeps QUIC off for 24 hours and logs "Several runs in a row with QUIC enabled ended early, so QUIC stays off until the cooldown ends". Later starts within those 24 hours log "QUIC stays off: several runs in a row with QUIC enabled ended early and the cooldown has not ended". The count clears once a run passes 10 minutes ("The run has passed the QUIC short-run limit; the count is clear"), on Ctrl+C or an auto-update restart, and when a run starts more than 24 hours after the one before it. Three runs in a row that each end within 10 minutes for ordinary reasons also count and can trip it, for example stopping or restarting from the tray, `freenet service restart`, or shutting down or logging off soon after each logon. `quic-ignore-crash-breaker = true` (`--quic-ignore-crash-breaker`, `FREENET_QUIC_IGNORE_CRASH_BREAKER=1`) overrides the breaker on every platform; a flag or environment

value is saved to `config.toml` and stays until you remove it there. On Windows, starting with the override set erases the trip, so removing the override later does not bring the 24 hours back. The protection that rolls back a crash-looping update is not there on Windows, so keep QUIC `off` there unless you are testing it.

`allowed-host` on a loopback bind (since v0.2.155). When the node's API listens only on localhost (the default), it no longer adds the addresses the machine's hostname resolves to (such as its LAN address) to the accepted `Host` names. This skips a blocking hostname lookup at startup, which kept the API from listening for about 35 seconds on one macOS 26 machine. If you reach a loopback-bound node through a forwarder that keeps the original `Host` header (for example `192.168.1.20:7509`), add that address with `--allowed-host 192.168.1.20` (it can be given more than once). Otherwise the node rejects those requests.

6.3 Ports and firewalls

- **7509/TCP, localhost only** — dashboard and WebSocket API. Do **not** expose this to the internet; it is meant for your own browser and local tools.
- **31337/UDP** — peer traffic. Freenet's transport performs NAT traversal, so most home users need no router configuration. Opening/forwarding 31337/UDP can improve connectivity but is not required unless you run a gateway.

6.4 Running a gateway (advanced)

A gateway is a node with a stable public address that helps new peers join. To run one you need a public IP and an open UDP port, and you start the node with:

```
freenet network --is-gateway \
    --public-address <your.public.ip> --public-port 31337
```

Gateways run "isolated" — they do not themselves bootstrap through other gateways. Operating a public gateway is a service to the network; if you're interested, coordinate with the project through Matrix so your gateway can be listed in the official index.

Gateways are named by role now (v0.2.134). Documentation, configuration examples, and published keys use `gw1.freenet.org`, `gw2.freenet.org`, … rather than the machine names (`vega.locut.us`, `nova.locut.us`) used before, so a gateway can move hosts without every example going stale. Public keys follow the same scheme (`https://freenet.org/keys/public.gw2.pem`). This is a naming change only: your node still discovers gateways automatically from `https://freenet.org/keys/gateways.toml`, and nothing in your config needs editing. Where you see an old host name in a third-party guide, expect the role name instead.

Part IV — The Operational Self-Check

Run this whenever you want to confirm your node is healthy — after installing, after an upgrade, after a reboot, or just periodically. The whole check takes about two minutes. Each step says what **good** looks like and where to go if the step fails.

7. The Ten-Step Health Check

Step 1 — What am I running?

```
$ freenet --version
```

✅ Prints a version (e.g. `0.2.123`) with a git commit and build timestamp. ❌ "Command not found" → the install directory (default `~/.local/bin`) isn't on your `PATH`, or Freenet isn't installed. → §2.

Step 2 — Is the service healthy?

```
$ freenet service status        # add --system for a system-wide install
```

✅ Service installed, active/running, recent start time, paths listed. ❌ Not installed → §2.2 / `freenet service install`. ❌ Installed but stopped → `freenet service start`, then re-check. If it stops again, go to §11.1.

Step 3 — Does the node answer locally?

Open `http://127.0.0.1:7509/` in a browser, or:

```
$ curl -sf http://127.0.0.1:7509/ >/dev/null && echo OK
```

✅ Dashboard loads / `OK`. ❌ Connection refused → the node isn't running or the WS port was changed → §11.2.

Step 4 — Is it the version I think it is?

Compare `freenet --version` (the binary on disk) with the running service's version shown in `freenet service status`. ❌ They differ → the node is running an old binary; restart the service, and if it persists run `freenet service doctor` (§11.3).

Step 5 — Are there peers?

```
$ fdev query
```

✅ Shows open connections to other peers. ❌ No connections after several minutes → §11.4 (connectivity). *(No `fdev`? It's optional — the dashboard also reflects network state, and you can skip to Step 6.)*

Step 6 — Anything alarming in the logs?

```
$ freenet service logs --err
```

✅ Quiet, or only transient warnings. ❌ Repeated errors — note the message and match it against §11's symptom table.

Step 7 — Am I up to date?

```
$ freenet update --check
```

✅ "Already up to date" (or it names a newer version — see Part VI to decide). Supervised installs update themselves; if this reports you are several versions behind *and* you installed the service, something is blocking updates → §11.3.

> ⚠️ **This step matters more than it used to** (v0.2.133): the network now enforces a minimum compatible version at the connection handshake. A node that falls too far behind isn't just missing features — every peer refuses its connections and it is cut off entirely. See §13.

Step 8 — Disk headroom.

```
$ df -h ~/.local/share/freenet
```

✅ Free space comfortably above 1 GiB. Freenet bounds its own hosting storage (§17), but logs and databases still need room.

Step 9 — Clean restart survives. *(optional but recommended after upgrades)*

```
$ freenet service restart && sleep 10 && freenet service status
```

✅ Comes back to running; dashboard loads again.

Step 10 — Deep diagnostics. *(optional)*

```
$ fdev diagnostics
```

✅ Prints detailed node state: network info, subscriptions, metrics, uptime. This is the same information a diagnostic report captures (§10).

> **Make it a habit:** steps 1–3 alone (version, service status, dashboard) catch the vast majority of problems and take under 30 seconds.

8. Reading the Signals

Three numbers the node uses to talk to its supervisor are worth knowing, because you will see them in logs and `systemctl` output:

Exit code	Meaning
`0`	Clean, requested shutdown. Normal.
`42`	"Restart me." Usually this means a new release is available, and the supervisor runs `freenet update` and restarts. It is also used when the node's network listener stops for good. During an upgrade this is **normal and healthy**. If it repeats without an upgrade, see §11.1.
`43`	"Another Freenet node is already running" (port conflict). The wrapper resolves genuine orphan conflicts itself; if you see this repeatedly, see §11.2.

9. Correct what needs correcting — the principle

The self-check is designed so that **every failing step points at a fix**:

- Steps 1–2 fail → installation problem → Part II.
- Steps 3–4 fail → service/binary problem → §11.1–11.3, `freenet service doctor`.
- Step 5 fails → network problem → §11.4.
- Step 7 fails → update pipeline problem → Part VI.
- Anything you can't classify → generate a diagnostic report (§10) and ask for help (§12).

10. Diagnostic Reports

When you need help — or want a snapshot of node health for your own records — generate a diagnostic report:

```
$ freenet service report
```

The report bundles: system info (OS, architecture, hostname), exact version and build info, recent logs (main and error), your config file, and — if the node is running — its live network status queried over the local WebSocket API. You'll be prompted for a one-line description of the problem, then the report is uploaded to the Freenet diagnostic server and you get a reference to share in chat or an issue.

Useful variants:

```
$ freenet service report --local report.json   # save locally, upload nothing
$ freenet service report -m "node has no peers since yesterday"
$ freenet service report --no-message          # skip the description prompt
```

Privacy note: the report contains your logs and config. Use `--local` first if you want to inspect exactly what would be sent. Since a report uploads your logs wholesale, v0.2.134 added log-side redaction of credentials that external services can echo back: an authorization token or key returned in a collector's error body is stripped before that body reaches the log, and any `user:password@` embedded in a configured endpoint URL is stripped from every log line. That protects the metrics-collector credentials described in §19 — it is not a licence to put secrets in your config, which is uploaded as-is. If the node isn't reachable when the report is generated, the report says *why* (connection refused, timeout, …) rather than silently omitting network status — that distinction itself is a useful diagnostic.

Where reports go, and uploading from behind a corporate proxy (v0.2.143). Reports now upload to `telemetry.freenet.org`, the same role-based name the telemetry endpoint moved to in v0.2.134 (older releases still use the previous address, which the server keeps open for them). Uploading a report is also the one thing your node now does that **trusts your operating system's certificate store** as well as Freenet's bundled one. That lets `freenet service report` work on networks where an administrator inspects HTTPS through a proxy whose certificate is installed on the machine. It is deliberately limited to reports: `freenet update` and auto-update still accept only the bundled certificates, so on such a network they keep failing until release signatures are mandatory. If an upload fails, the error now ends with the fallback: re-run with `--local <PATH>` and send the file privately to a Freenet developer — for example as a direct message on Matrix, not in the public room, since it holds your config and recent logs.

Part V — Correcting Problems

11. Troubleshooting by Symptom

11.1 The node won't start, or keeps stopping

1. `freenet service logs --err` — read the last few lines; the node states its reason for exiting.
2. Check for the **disable marker**: if you (or someone) ran `freenet service disable`, the node refuses to run by design. Fix: `freenet service enable`.
3. Crash loops are handled with backoff by the supervisor (10 s doubling up to 5 min). If crashes started **right after an update**, do nothing for a few minutes: the node counts probation crashes and **rolls itself back automatically** to the previous known-good version (§14.2). If crashes are unrelated to an update, capture a report (§10) and ask for help (§12).

11.2 Port already in use / exit code 43 / dashboard unreachable

> **Restarting immediately after a stop is more reliable in v0.2.136.** The check for "is another node already running?" used to be fooled by the socket of a process that was still dying, so a restart issued a moment after a stop could fail with a port conflict against a node that no longer existed. The check now retries past that. If you previously had to wait a few seconds between `stop` and `start`, you no longer do.

- Another process (often an orphaned old Freenet process) holds the network port. The supervisor kills genuine orphans automatically (up to 3 attempts); if the conflict persists, a *different* application owns the port — find it (`ss -ulpn | grep 31337` on Linux) and either stop it or move Freenet with `--network-port`.
- **Since v0.2.144 the Linux service also clears a leftover node on the *same* version.** Before, its start-up check only stopped an orphaned `freenet network` running an *older* release. A same-version leftover — for example a node you started by hand that kept running after you logged out — was spared, so the service started, found the port taken, and stood down, leaving that unsupervised node running. Now any orphaned `freenet network` of your own user is stopped first, whatever its version, except one started with its own `--ws-api-port` or `--config-dir` (a deliberately separate instance) or one that another systemd service is running. The log line to look for is `freenet pre-flight: stopping orphaned node <pid>`. An existing unit picks this up on its own: the next update rewrites a unit file that differs from the current template.
- Dashboard specifically unreachable while the node runs: confirm the WS API port (default 7509) wasn't customized, and that you're browsing from the same machine — the API binds to localhost.

11.3 Node seems stuck on an old version

Symptoms: `freenet update --check` says a newer version exists for days; or `freenet --version` (binary) disagrees with the running service.

```
$ freenet service doctor      # add --system for system-wide installs
```

`doctor` re-points the service at the current binary, reaps stale orphaned `freenet network` processes still holding the port on an old binary, and restarts cleanly. It fixes the classic "frozen on an old version" state that a plain `restart` cannot. After running it, re-do self-check steps 2–4 and 7.

Also remember: **unsupervised** nodes (installed with `FREENET_NO_SERVICE=1`, or hand-run with `freenet network`) do not auto-update at all — see §14.3. And **dev/dirty builds** never auto-update by design.

Node restarting by itself on Windows (fixed since v0.2.146). Before v0.2.146, some ordinary network events could stop a node's network listener and make it exit and restart. On Windows this included an oversized packet or a network reset caused by a remote host. The log showed `Fatal UDP recv_from error, listen task exiting`, and the exit code was usually 42, so it looked like an update. Since v0.2.146 the node drops the bad read and keeps running, and logs a warning instead: `UDP recv_from error, dropping the read and continuing`. To keep the log readable, it writes at most 5 of these warnings a minute for each class of error and reports how many it skipped. Errors caused by remote peers never stop the node. It exits only when its network socket really is unusable, or when the socket has kept failing with unrecognised errors (at least 1000 of them, with no successful read) for at least a minute. In both cases the supervisor restarts it as before. There is nothing to configure. If you still see the "Fatal" line after updating, capture a report (§10) and ask for help (§12).

A bad packet from a peer no longer crashes the connection (since v0.2.148). Before v0.2.148, two kinds of packet from another peer could make your node panic while reading from that peer. One was a packet too short to be valid. The other was a packet that came from a real connected peer but that your node could not read, for example one using a message format from a newer Freenet release. Your log showed a panic message and your node lost its connection to that peer. While newer and older versions were mixed on the network, this could keep happening. The rest of the node kept running. Since v0.2.148 there is no panic. A packet that is too short is treated like any other packet that fails decryption: it is dropped quietly, but many of these in a short time can still close the connection. An unreadable packet from a connected peer is dropped, the connection stays open, and the node writes one warning per connection: `Dropping authenticated packet that does not decode; later ones on this connection log at debug`. The warning names the peer's address and version. It is harmless and there is nothing to configure. If you see it for many different peers, your node may be out of date: see §11.3.

11.4 No peers / can't join the network UPDATED

0. **Check your version first** (v0.2.133): the transport handshake enforces a minimum compatible version, so a node that is too far out of date is *refused by every peer* — it looks exactly like a connectivity problem but is fixed by `freenet update`. Self-check step 7 rules this out in seconds.
1. Give a fresh node a few minutes — joining requires a round-trip through a gateway.
2. Check basic connectivity: can you reach the internet at all? Does your network block outbound UDP? (Some corporate/university networks do.) Freenet requires outbound UDP; without it the node cannot join.
3. Check the log for gateway errors (`freenet service logs`). The node fetches the gateway index from `https://freenet.org/keys/gateways.toml`. If that fetch fails, the log shows `Remote gateway fetch failed, falling back to local cache` and the node uses the cached `gateways.toml` in your config dir. Since v0.2.156 it also adds the official gateways built into the binary (gw1.freenet.org and gw2.freenet.org) that your cache does not already list, and logs `Remote gateway index unavailable; added compiled-in fallback gateways` when it adds any; a cached entry wins if both name the same address. So a brand-new node can start while the index is down, where older versions refused to start. The built-in list is never used with `--skip-load-from-network` or in local mode.

Also since v0.2.156, one bad gateway no longer stops the node. It skips a malformed index entry, a gateway whose public key is missing, unreadable or invalid, and a gateway whose hostname does not resolve, logs a warning for each (for example `Skipping malformed gateway entry in the remote index`, `Skipping gateway whose public key is not valid hex` or `Skipping gateway whose address cannot be resolved`), and carries on with the rest. A skipped gateway is not retried until the next restart. If a gateway's key can't be downloaded this time but a valid copy was saved earlier, the node uses that copy (`Using the previously cached key for this gateway`), and as long as at least one gateway from the index worked it also keeps that gateway's cached entry in `gateways.toml`. A few of these warnings are harmless. A regular (non-gateway) node fails to start only when no usable gateway is left: with `no configured gateway address could be resolved` if every gateway failed DNS, or otherwise `no configured gateway could be used (N skipped): ...` followed by up to five reasons. If you see either, check your DNS and network and any gateway key files you edited by hand, then ask in Matrix (§12). 4. **Did you ever run `freenet local` on this machine?** (fixed since v0.2.145) Local mode and network mode use the same `config.toml`. Older `freenet local` runs saved a loopback address there as `network-address` (`::1`, or `127.0.0.1` in early releases). The next `freenet network` boot read it back and listened only on loopback, where no gateway can reach it, so the node never found any peers. Since v0.2.145, a normal (non-gateway) node ignores a stored `network-address` of exactly `::1` or `127.0.0.1`. It listens on all interfaces instead (`::`, or `0.0.0.0` for an IPv4 value) and fixes `config.toml` for you. On that one boot the log shows a warning that starts `Ignoring network-address ... from config.toml`. You don't need to do anything. If you really do want a loopback bind, for example on a private test network, pass `--network-address` (or set `NETWORK_ADDRESS`) together with `--skip-load-from-network`, both after the subcommand. `--skip-load-from-network` is remembered once set; the address flag is not and has to be passed every time. Gateways, and any other loopback address such as `127.0.0.5`, are left alone. 5. Still stuck → diagnostic report + ask in Matrix (§12).

11.5 "No file descriptors available" or resource errors

The node raises its own file-descriptor limit at startup to the kernel hard limit automatically. If you still see `EMFILE`/fd errors on an unusual setup, raise the hard limit for the service (systemd: `LimitNOFILE=`), then restart.

11.6 Disk usage growing

Hosted contract state is bounded (default 1 GiB; overall disk budget is additionally capped — §17) and evicted least-valuable-first. Since v0.2.130, **logs are bounded too**: the node prunes its own log directory to a 512 MiB budget in the background, overridable with the `FREENET_LOG_DIR_MAX_BYTES` environment variable (shrink it on a quiet peer to hand back disk, or widen it while chasing an intermittent fault). The compiled-WASM cache is also now bounded by disk headroom, not just RAM (v0.2.126). If disk still grows unboundedly, check the data directory with `du` and file an issue — nothing is supposed to grow without a budget anymore.

Disk the "Disk used" figure leaves out (v0.2.144). On the dashboard's hosting card, hover over **Disk used** for its breakdown. Besides state, WASM and the compile cache — what the figure counts against the disk budget — it now lists two things your node measures but does **not** count:

- **The database file.** Contract state lives inside it, so it is normally larger than the state figure; the tooltip says by about how much. That difference is dead space, other tables, database overhead, and any backup left by a migration. A large gap explains a `du` total well above "Disk used".
- **The webapp cache**, which is shared by every node run as the same user.

Either reads "not measured" until the node has first looked at it. Neither is charged against the disk budget yet, so a budget alone does not bound them.

11.7 An app loads, but parts of it are missing

Since v0.2.137 an app asset your node cannot serve comes back as a plain **404** instead of a 500 carrying an operating-system error message in the page. That is the correct answer to give a browser, and the response is explicitly not cacheable (`Cache-Control: no-store`), so once the asset is available a normal reload picks it up rather than a stale error sticking around. The trade-off is that the browser can no longer tell you *why* it is missing.

Two quite different causes look identical from the browser, and are separated only in the node's own log:

1. **The asset genuinely isn't there yet** — the app's contract hasn't finished synchronizing. Reload after a moment; the health check (§7) tells you whether the node is actually connected.
2. **The asset is there but unreadable** — wrong ownership or permissions on the webapp cache directory, or a damaged cache. In this case, and only this case, your node logs a warning naming the path:

```
bash freenet service logs --err # or search the whole log for the affected area freenet service logs | grep -i webapp
```

A permissions warning means the file exists and your node was refused it: fix ownership of the data directory (§6.1) and restart. If the cache itself looks damaged rather than mis-owned, `freenet service doctor` (§11.3) repairs a wedged install.

If no warning appears at all, treat it as case 1 and give the contract time to arrive.

11.8 An app still shows its old version after an update

App publishers can release a new version of an app at the same address. Before v0.2.146 your browser could keep its own copy of the app's main page, so an open tab, or even a normal reload, could keep running the old version, sometimes until you cleared the browser cache.

Since v0.2.146 your node tells the browser never to store an app's pages (`Cache-Control: no-store` on every HTML page under `/v1/contract/web/<key>/` and `/v2/contract/web/<key>/`). A normal reload now always fetches the current version of the page. There is nothing to configure.

If an app still looks partly old after a reload, the cause is most likely its scripts, styles or images. This change does not cover those, so your browser may still use a copy it saved earlier. Apps that give each new build's files new names are not affected. If you see this, do a hard reload (Ctrl+Shift+R, or Cmd+Shift+R on macOS) or clear the site's cached data for your node's address. If the page itself is still old after a normal reload, your node may not have received the update yet. Wait a moment and reload again.

12. Getting Help

1. **Generate a diagnostic report first** (§10) — it answers 90% of the questions a helper would ask.
2. **Matrix chat:** `#freenet-locutus:matrix.org` — the developers are active here. (Not `#freenet:matrix.org` ; older documentation linked that room by mistake.)

3. **GitHub issues:** https://github.com/freenet/freenet-core/issues — for reproducible bugs. Include your report reference, `freenet --version` output, OS, and what the self-check showed.
4. **freenet.org** — links to current docs, FAQ, and community channels.

Part VI — Upgrading

13. How Auto-Update Works

> **Staying current is no longer optional** (v0.2.133). Freenet ships releases frequently — sometimes several a day — and peers are expected to converge on new releases within hours. The network enforces a `min-compatible-version` floor **as a hard gate at the transport handshake**: once a node drops below the floor, every peer refuses its connections and it is cut off. The supervised auto-update pipeline below is what keeps that from ever happening to you.
>
> **The floor rose to v0.2.122 in v0.2.143.** Releases v0.2.120 and v0.2.121 cannot update themselves, so the network now refuses them. A node still on either of those has to be updated by hand once (§14); from v0.2.122 on, the pipeline below takes over again.

Freenet releases frequently, and the update system is designed so a supervised node **keeps itself current with zero attention from you** — while protecting you from a bad release. The pipeline:

1. **Detection.** The running node notices a new official release and exits with code **42** ("update me"). It never overwrites itself while running.
2. **Download & verify.** Since v0.2.143 the node **downloads the release before it exits 42**, while it is still serving, into a cache under its state directory (it keeps retrying for up to two hours, resuming a partial download). The supervisor then catches exit 42 and runs `freenet update`, which installs from that cache — or downloads the release itself if the cache is missing or fails any check. Either way the release is checked against its `SHA256SUMS.txt` manifest, and **the manifest's ed25519 signature is verified against a public key baked into your binary** — a cached download gets exactly the same checks as a fresh one. An artifact that fails verification is never installed. *Why the move:* the install step runs inside a short shutdown time limit, and on a slow connection the old in-limit download could be killed part-way, restart the old version, and repeat forever. Only a node already on v0.2.143 or later gets this; one on an older release updates the old way one last time.
3. **Swap & restart.** The verified binary replaces the old one (which is kept as the known-good fallback) and the service restarts on the new version.
4. **Probation.** For a short period after an update the node is "on probation": crashes are counted.
5. **Automatic rollback.** If the new version crash-loops during probation, the updater **restores the previous known-good binary**, pins the bad version as known-bad on this node (so it won't be retried), and restarts. This works even with no network connection.

You can watch all of this happen in `freenet service logs` — a healthy update shows an exit-42, an update run, and a restart on the new version.

When the safety net itself fails, the updater now says so (v0.2.135). Steps 4 and 5 above depend on two things that can quietly go wrong: saving the known-good binary before the swap, and arming the probation marker after it. Either one failing used to be silent — the update landed, everything looked normal, and rollback protection was simply *off*. Both now print a warning:

```
Freenet: failed to snapshot the known-good binary for crash-loop rollback: …
PROCEEDING WITHOUT ROLLBACK PROTECTION for this update — if this version
crash-loops it will NOT be auto-reverted. Check the permissions and free space
on the Freenet state directory.
```

```
Freenet: installed the update but FAILED TO ARM crash-loop rollback protection: …
If this version crash-loops it will NOT be auto-reverted. Check the permissions
and free space on the Freenet state directory.
```

Both point at the same two causes: **permissions** on the Freenet state directory, or **free disk space**. Fix whichever applies and run `freenet update --force` to redo the update with protection in place.

Note that `--quiet` does **not** suppress these. The flag means "be less chatty", never "stop recording safety events" — a suppressed warning here is exactly the case where you would never learn that the net was missing until a release crash-looped with nothing to roll back to.

The end of probation now announces itself, accurately (v0.2.136). When the probation window closes the node prints what actually happened, and each possible outcome gets its own wording rather than a line shared between two of them — because the two facts you would act on (did this version pass, and is rollback still available?) differ between outcomes, so a shared line is wrong for at least one of them.

The same release fixed something worth knowing about, because it is an unusually neat failure. The announcement used to be printed with a call that *panics* if the write fails — which it can, when the supervisor's reader has already gone away. A panic on a worker thread would exit the node non-zero; a non-zero exit during probation is scored as a crash; and three crashes roll the node back. So the announcement that an update had **succeeded** could itself have caused the rollback it was reporting as no longer necessary. It now writes that line without being able to take the process down.

When an update cannot be downloaded, the node stays up (since v0.2.149). Before this, a node that could not fetch a new release (GitHub slow, throttled or blocked) looped: it exited 42, the download failed, it restarted on the old version, saw the same release and exited again, every few minutes. Now a failed download puts that release on a cooldown and the node keeps serving on its current version:

- The cooldown is about 1 hour after the first failure and about 2 hours after the second (each ±20%).
- After the third failure in a row, the node exits 42 anyway so that `freenet update` can try its own download. It also exits at once if it cannot record the failure.
- Failures more than two days old stop counting.

There is also a hard limit of 320 MiB of release downloads per node in any 24 hours. If that budget runs out, the release is held, without counting as a failure, until the budget resets; the log shows "Update deferred: the download budget is spent", and `freenet update --force` overrides the hold.

`freenet service report` shows the current state, for example "Update to 0.2.NNN failed 2 time(s) in a row (next attempt in 95 min). Last error: …", and the log shows "Staying on the current version and trying the update again after a cooldown". A node a few hours behind the latest release while that line keeps appearing is working as intended. If it persists, check that the node can reach github.com.

14. Updating Manually

14.1 Commands

```
$ freenet update --check    # just tell me; change nothing
$ freenet update            # download, verify, install the latest release
$ freenet update --force    # reinstall even if already on the latest
$ freenet update --quiet    # no interactive output (for scripts)
```

After a manual update, restart the service (`freenet service restart`) and run self-check steps 1–4.

14.2 After a bad update

Normally you do nothing: automatic rollback (§13 step 5) handles a crash-looping release, and the node then simply skips that version. When a fixed release ships, update as usual — `freenet update --force` if the fixed release reuses a version number the node has pinned as known-bad.

First check whether rollback protection was actually armed. Since v0.2.135 the updater warns when it could not snapshot the known-good binary, or could not arm the probation marker (§13). If you see either warning in `freenet service logs` , automatic rollback is *not* available for that update and a crash-looping version will keep crash-looping. Recover by hand:

1. Fix the cause the warning names — permissions or free space on the state directory.
2. Reinstall a known-good version: `freenet update --force` , or install a specific release with `FREENET_VERSION=<version>` via the installer (§2.2).
3. Restart and run self-check steps 1–4.

If auto-update says it is LOCKED OUT. After **three consecutive failed update attempts**, the node stops trying and logs a warning that begins *"Auto-update is LOCKED OUT"*, naming the state directory it is using. Earlier editions of this manual did not document this lockout at all — they should have, because until v0.2.140 it was **permanent**: a node that ran stably and never restarted would never ask again, and would drift below the network's compatibility floor (§7 step 7) with nothing more than one log line to show for it.

Since **v0.2.140** the lockout expires: a locked-out node retries **at most once a day**, so once whatever broke its installs is fixed, it recovers by itself. Since v0.2.149 that daily retry also waits for any download cooldown and for the download budget to reset (§13). To update immediately and reset the counter, run `freenet update` yourself. If updates keep failing, the most common cause is a binary path that the service account cannot write to — for example, a hand-installed binary the service user doesn't own. (On NixOS, see the `STATE_DIRECTORY` note in §2.7.)

14.3 Keeping an *unsupervised* node current

A node you run by hand (`freenet network` in a terminal, or installed with `FREENET_NO_SERVICE=1`) will detect an update, print that it needs one, and exit — and then nothing restarts it. Your choices:

- run `freenet update` yourself when prompted in the logs, then start the node again; or

- (better) convert to a supervised install once: `freenet service install`.

15. Upgrading *Yourself*: as you advance

The manual's growth path (About This Manual) mirrors a real progression:

1. **Newcomer → Everyday user:** make the self-check (Part IV, steps 1–3) a monthly habit; learn to read `freenet service logs`.
2. **Everyday user → Operator:** learn `service doctor`, diagnostic reports, and the update pipeline; skim your config (§6) so defaults are choices, not accidents.
3. **Operator → Power user:** take control of secrets (§16) and resource budgets (§17); consider running a gateway (§6.4).
4. **Power user → Developer:** install `fdev` and work through the developer guide (§18) and the freenet-ping example app.

Each new manual revision highlights what's changed (badges + Appendix E), so re-skimming the manual after an upgrade shows you exactly which capabilities are new since you last read it — your growth chart and the software's, on the same page.

Part VII — Advancing: Power Use

16. Secrets: Protecting Your Keys and Private Data

Delegates keep your private data (identity keys, chat-room keys, credentials) **encrypted at rest** on your node. The design in one paragraph: every secret is encrypted with a per-delegate key (DEK), and every DEK is derived from a single master **Key Encryption Key (KEK)** held in a pluggable backend. Manage it with the `freenet secrets` commands:

Command	Purpose	Node state
`freenet secrets status`	Show the active KEK backend and the KEK fingerprint (never the key itself).	any
`freenet secrets provision`	Choose a KEK backend **before first start** — e.g. opt into the OS keyring.	before first run
`freenet secrets rotate`	Generate a new KEK and re-encrypt every secret and snapshot under it.	**stopped**
`freenet secrets migrate`	Move the KEK between backends (`file` ↔ `keyring` ↔ `systemd`).	**stopped**
`freenet secrets snapshots`	List per-secret backup history (metadata only — never plaintext).	any
`freenet secrets restore`	Roll a secret back to an earlier snapshot (the current value is snapshotted first, so this is reversible).	**stopped**
`freenet secrets export`	Pack a scope's secrets into one **encrypted, portable bundle** — e.g. to move your identity to a new machine.	**stopped**
`freenet secrets import`	Import a bundle produced by `export` on another node.	**stopped**

Delegates that hold your secrets are harder to wedge (v0.2.135). Delegate WebAssembly now runs under the same **wall-clock time limit and panic capture** that contracts already had, so a delegate that hangs or panics is stopped and reported instead of tying up the node — and delegate execution was moved off the node's serial loop, so a slow delegate no longer blocks unrelated work. A related fix means each secret is **decrypted once per read** rather than repeatedly, which shortens the window in which plaintext exists in memory. Nothing changes in how you use `freenet secrets`; the guarantees underneath it got stronger.

Backends: `keyring` (OS keychain / Credential Manager — strongest for desktops; the auto-resolver deliberately does *not* pick it silently, because it can trigger an OS consent prompt: opt in with `provision`), `systemd` (systemd-credentials — good for Linux servers), `file` (portable default).

Moving to a new machine: stop the node → `secrets export` → copy the bundle → install Freenet on the new machine → `secrets import` → start. The bundle is always encrypted (passphrase-derived key), so it is safe to carry on a USB stick — but treat it like the keys it contains.

Make backups a habit, not just a migration step. Your delegate keys are the one thing on your node that cannot be re-fetched from the network — if the disk dies and there is no bundle, identities and room keys are gone permanently. Run `secrets export` after creating any identity you care about and after any significant new secret, and store the bundle (plus its passphrase, separately) somewhere that doesn't share fate with the machine. Everything else — hosted contract state, caches, the binary — is replaceable; the secrets are not.

Full operator documentation: `docs/secrets-at-rest.md` in the source repository.

17. Storage and Resource Tuning

Your node hosts a share of the network's contract state. Three dials bound it:

Dial	Default	Notes
`--max-hosting-storage`	**memory ÷ 8**, clamped 128 MiB–1 GiB	Budget on tracked contract state **kept on disk** (only memory-bounded caches of it live in RAM); beyond it, least-valuable contracts are evicted and their disk reclaimed.
`--hosting-disk-pct`	0.5	Fraction of the disk capacity available to Freenet used to size the aggregate disk budget. The effective budget is the **minimum** of the two budgets.
`--max-hosting-disk`	32 GiB	Hard cap on the disk budget regardless of disk size.

Other useful dials:

- `--module-cache-budget-bytes` — cache of compiled contract WASM. Default scales with RAM (RAM/8, clamped 64 MiB–4 GiB). Raise it on a busy node with many contracts to avoid recompilation churn.
- `--max-blocking-threads` — WASM execution parallelism (default 2×CPU cores, clamped 4–32).

Since v0.2.126–0.2.127 the node bounds overall peer memory (~2 GiB cap) and bounds the compiled-WASM cache by actual disk headroom as well as RAM.

> **Correction.** Earlier editions of this section said that since v0.2.126 the node sized hosting eviction from *"live memory measurements rather than a hardcoded per-contract overhead estimate."* That overstated it. Until **v0.2.141**, the RAM each hosted contract cost was charged at a **fixed estimate of 1 MiB per contract**. Only from v0.2.141 does the node count what hosted contracts actually hold in memory.

The RAM side has its own dial: `--hosting-mem-share` (v0.2.141). Hosting costs memory as well as disk — mostly the summaries neighbouring peers send so your node can keep each hosted contract current. Two things changed in v0.2.141:

- **It is measured, not estimated.** The node now counts those bytes, including storing summaries that several neighbours send identically only once, instead of assuming 1 MiB for every contract. A node holding many small contracts was previously charged far more than they cost.
- **The budget is a share of the node's memory limit.** That limit is physical RAM, or a smaller cgroup / systemd `MemoryMax` limit when one applies — so a container or a capped service is sized against what it is actually allowed. The default share is ⅛ (`0.125`), never below 64 MiB. Past the budget, the node stops hosting its least-demanded contracts.

The two dials are independent: `--max-hosting-storage` bounds state **on disk**, `--hosting-mem-share` bounds what hosting holds **in RAM**, and either one can be the limit that binds first. Raise `--hosting-mem-share` on a machine with memory to spare. The node warns at startup above `0.5`, because the node's other caches already take about a quarter of the memory limit and the runtime needs about a tenth; above one half there is little left for anything else.

Two corrections landed in v0.2.136, and earlier revisions of this manual had both of them wrong.

First, the default is not a flat 1 GiB. It is one eighth of the memory available to the node — system RAM, or the cgroup limit if that is lower — clamped to between 128 MiB and 1 GiB. 1 GiB is the ceiling, not the starting point, so a small VPS starts well below it.

*Second, this dial governs **disk**, not RAM.* Contract state is kept on disk; only bounded caches of it are held in memory. Calling it a "RAM-style budget", as this manual did through revision 1.3, misdescribed what you are actually contributing.

`--max-hosting-storage` is how you contribute disk to the network — and there is a rule for raising it. Keep it a few GiB *below* your disk budget. That budget is the smaller of `--hosting-disk-pct` of the space available to Freenet and `--max-hosting-disk`, and it counts WASM code and the compile cache (up to about 512 MiB) as well as state. The gap is the headroom those need. **If you set it too close to the disk budget, the node stops accepting new contracts and state growth instead of evicting to make room** — the opposite of what you intended by raising it.

Worked example: with 40 GiB available to Freenet and default settings, the disk budget is min(0.5 × 40 GiB, 32 GiB) = 20 GiB, so set `--max-hosting-storage` to about **17 GiB**. On a larger disk the 32 GiB cap binds unless you raise `--max-hosting-disk` too. Leave more headroom if your node hosts many contracts with distinct code, since WASM grows with those. Raising this dial does not make your node take on more contracts than its memory can hold — that ceiling is `--hosting-mem-share`, above.

The defaults are deliberately conservative: a stock node donates a bounded, predictable amount of your disk and memory. Raising the budgets makes your node a more valuable network citizen; it never grows unbounded either way.

18. First Steps as a Developer

Everything on Freenet — every app, every chat room — is contracts plus delegates plus a web front-end, and the tooling is a single CLI:

```
cargo install --path crates/fdev     # or: cargo install fdev
```

`fdev` command	Purpose
`fdev new`	Scaffold a new contract or web-app package.
`fdev build`	Build a contract/delegate/web-app to WASM.
`fdev publish`	Publish a contract or app to the network (or your local node).
`fdev query`	Show your node's open peer connections.
`fdev diagnostics`	Detailed node state: network, subscriptions, metrics.
`fdev inspect`	Compute a contract's ID without publishing.
`fdev verify-merge`	Check that a contract's merge obeys the laws the network requires (order-independence, associativity, idempotence) — the *same* verifier the network runs. A contract that fails here cannot converge on the network. Formerly named `conformance`. *(New in v0.2.129–0.2.133.)*
`fdev website init/publish/update`	Keypair-based publishing of static websites on Freenet.
`fdev commands get/subscribe/update`	Raw contract operations against a node's WebSocket API.

Associativity and capped contracts (since v0.2.147): a contract that keeps only a limited history, such as a chat room with a message cap, a feed or a log, can give slightly different results depending on how three updates are grouped when they are merged: merging A with B first and then the result with C, or merging B with C first and then A with that result. For example, one grouping drops an old message to make room, while the other keeps it. Since v0.2.147, `fdev verify-merge` and the dashboard's "Merge laws" card no longer report this as a violation, as long as the two results reach the same state after further merges (up to 4 passes). An associativity violation is reported only when the two results do not settle on one state, or when the contract rejects one of them as invalid. If the 4 passes run out before a verdict, the case is reported as inconclusive and does not count against the contract. If an earlier version flagged a capped contract (for example a River room) as "Violation — removal-eligible" for associativity, upgrade and check again. The flag was most likely a false positive.

Large state corpora (since v0.2.153): when you give `fdev verify-merge` more states than it can pair, it samples them, 24 by default. The last state you supplied and the largest state are always sampled, and the cases from your newest states are queued first, so if the `--max-cases` budget runs out it is cases from earlier states that are dropped (before v0.2.153 the budget dropped the newest states first). Alongside the existing counts of cases skipped for size and for the budget, the report now also says how many cases sampling left out. None of these skipped cases change the verdict unless every case was skipped. To sample more, pass `--max-states-paired N` (3 to 1000); generation time grows with the square of N, and more pairs compete for the same `--max-cases` budget. Raising `--max-case-bytes` lets triples of big states fill the associativity checks, so some triples of small states that ran before may no longer run.

Breaking for delegate authors in v0.2.136: the *synchronous* delegate contract-write host functions have been removed. A delegate that wrote contract state through them must move to the asynchronous path (the same one that carries the network GET/SUBSCRIBE described below). Rebuild and test any delegate that writes contract state before publishing against this release.

For delegate authors in v0.2.137: a request for a delegate a node does not have now fails with the typed `DelegateError::Missing(key)` in network mode — the same error `freenet local` already returned. Previously a networked client only got a generic `OperationError` carrying a string, so it could not tell "this node doesn't have that delegate" apart from "this node failed", which is exactly the distinction a migration probe across predecessor delegate versions depends on. Match on the typed variant rather than parsing error text. Those misses are also no longer logged at error level, so probe traffic no longer fills the error log you use for triage.

Delegate subscriptions are durable as of v0.2.137, within limits your app should not assume away: 4096 persisted, 512 re-established over the network per boot, and nothing migrated when you publish a new version of a delegate (a new delegate key is a new identity — re-subscribe on first run). §4 has the user-facing view of the same change.

New for delegate authors in v0.2.138: manifests and capabilities. A delegate that must run with no app tab open declares it in a manifest compiled into the WASM:

```
#[delegate(manifest(lifecycle = [..], capabilities = [Background]))]
```

The node reads the manifest at registration, asks the user once *in its own prompt* — you cannot author or style it — and stores the answer against the **app**, not against your delegate, so shipping a new delegate build does not re-prompt your users. Your delegate then receives `LifecycleEvent::Installed` once per node and `LifecycleEvent::NodeStarted` after each start, but only if your manifest lists that kind *and* the app holds the grant. Design for the grant being absent: that is the default, and a refusal lasts a week.

Three limits to build against: one delegate may be bound to at most **8 apps** (background delivery is enabled if any bound app holds the grant); unprompted runs are budgeted for both time and contract operations, per delegate and node-wide, and are refused with a retry-later once spent; and a delegate whose registered parameters exceed **64 KiB** receives no lifecycle events at all.

Periodic wake-ups, v0.2.139. Add schedules to the same manifest and receive `InboundDelegateMsg::WakeupFired { tag }`:

```
#[delegate(manifest(capabilities = [Background], wakeups = [sync = 3600]))]
```

They are declared in the manifest rather than requested at run time on purpose, and the reason is a compatibility one worth copying: a node that predates the feature skips an unknown JSON field and honours the rest of your manifest, whereas a new host import would have failed instantiation and a new outbound message variant would have failed decoding of the entire outbound batch. One delegate build therefore works on old and new nodes alike.

Build against the bounds: the interval is clamped to **60 s – 7 days**, tags are **1–64 bytes**, and a delegate gets at most **4** schedules. Only one fire per `(delegate, tag)` may be pending. Each fire re-checks that your manifest still lists the tag *and* that a bound app still holds the Background grant, so treat revocation as something that can happen between any two fires. Fires are metered by the same duty budget as lifecycle runs, and a fire that cannot run —

parked delegate, spent budget — is deferred and then given up rather than queued. Nothing about a pending fire is persisted; schedules are re-armed at node start, after a registration, and after a grant. Requires `freenet-stdlib` 0.12.1.

New for delegate authors in v0.2.135. V1 delegates can now issue **GET and SUBSCRIBE against the network**, not just against state the node already holds, and V2 delegate contract writes propagate to the network. Two constraints to design against: a delegate may hold at most **256 contract subscriptions**, and exceeding that **evicts the least-recently-notified** subscription rather than returning an error — so a delegate that quietly depends on a dormant subscription can lose it. The cap is intentionally not configurable, so you can rely on it being the same number on every peer. Delegate WASM now also runs under a wall-clock backstop with panic capture, so an infinite loop in your delegate surfaces as a reported failure instead of a hung node. This release also moves to **`freenet-stdlib` 0.10.0** — rebuild against it before publishing.

The recommended path: run a node in **local mode** (`freenet local` — a sandboxed node with no network traffic), then build and publish against it, then graduate to the real network. Study the example app `apps/freenet-ping` in the source tree — a minimal but complete contract + client that exercises the full put/subscribe/update cycle. The developer-facing architecture documentation lives in `docs/architecture/` and the whitepaper.

19. Monitoring: Exporting Your Node's Metrics

New in v0.2.134. Your node can export its own metrics — transport, ring, contract queue, memory — to any OpenTelemetry (OTLP/HTTP) collector you run: Prometheus with the OTLP receiver, Grafana Alloy, the OpenTelemetry Collector, or a hosted service. This is how you graph your own node over time instead of reading a number off the dashboard.

It is **off by default**, and it is a different thing from `telemetry-enabled` (§6.2), which sends a small amount of data to the project's own dashboard. Turning one on or off has no effect on the other; they share no setting.

19.1 Turning it on

In `config.toml`:

```
otel-telemetry-enabled = true
otel-endpoint = "http://collector.example:4318"
```

or on the command line:

```
freenet network --otel-telemetry-enabled \
    --otel-endpoint http://collector.example:4318
```

`--otel-telemetry-enabled=false` turns it back off without editing the file, and `FREENET_OTEL_TELEMETRY_ENABLED` works as an environment variable (unlike a plain flag, it honors `=false`). Nodes started with `--id` — test networks and the integration harness — never export, whatever the configuration says.

`config.toml` key	Default	Meaning
`otel-telemetry-enabled`	`false`	Enable the exporter
`otel-endpoint`	none	Collector base URL; `/v1/metrics` is appended for you
`otel-auth-mode`	`disabled`	`disabled` sends no `Authorization` header; `freenet` sends a signed token (§19.3)

19.2 Endpoint gotchas worth knowing before you debug

The standard OpenTelemetry environment variables take priority over `otel-endpoint`, and three of them bite:

- **The two endpoint variables are not interchangeable.** `OTEL_EXPORTER_OTLP_METRICS_ENDPOINT` is used exactly as written and must include the full path (`http://collector:4318/v1/metrics`), while `OTEL_EXPORTER_OTLP_ENDPOINT` takes the base URL only (`http://collector:4318`). Getting them backwards produces a 404 on every export.

- **The timeout variables are in milliseconds.** `OTEL_EXPORTER_OTLP_TIMEOUT=10` means 10 ms, and every export will time out. The node warns about implausibly small values.
- `OTEL_EXPORTER_OTLP_COMPRESSION` **is not supported.** Setting it makes the exporter fail to start, leaving you with no metrics at all; the startup warning names the cause.

Always include the scheme: a bare `collector:4318` parses as a URL but cannot be sent. When an environment variable overrides your configured endpoint, the node says so at startup, and the "OTel metrics exporter started" log line always names the endpoint actually in use — check that line first when nothing arrives. With no endpoint configured anywhere, the exporter targets `http://localhost:4318`. The default export interval is 60 seconds (`OTEL_METRIC_EXPORT_INTERVAL`).

19.3 Authentication, and why credentials refuse to travel in cleartext

For most setups leave `otel-auth-mode` at `disabled` and carry whatever your collector wants in `OTEL_EXPORTER_OTLP_HEADERS`:

```
OTEL_EXPORTER_OTLP_HEADERS="Authorization=Basic $(printf 'user:pass' | base64)"
```

The node never overwrites an `Authorization` header you set this way, so ordinary header auth works in the default mode with nothing Freenet-specific.

Whichever kind of credential is in play, the node refuses to leak it:

- **A credential requires** `https://` unless the collector is on loopback. Plaintext `http://` to anything other than `localhost` / `127.0.0.0/8` / `::1` fails the export with a warning rather than putting the credential on the wire. (The loopback exemption covers the common case, a collector sidecar on the same host.)
- **Redirects are not followed**, so an `https` endpoint cannot bounce your header to an `http` one.
- `HTTP_PROXY` / `HTTPS_PROXY` **are ignored.** Otherwise a proxy set in your environment would receive an export aimed at `http://localhost:4318` — passing the loopback exemption and leaving the machine anyway. If your collector is only reachable through a proxy, point `otel-endpoint` at the proxy.
- **Credentials are redacted from logs** (see §10).

`otel-auth-mode = "freenet"` is for collectors that verify Freenet node identities: it adds a bearer token signed with your node's transport key, so the collector can confirm the metrics came from the node they claim to. Enable it only for a collector that actually checks those tokens — it ships a signed assertion of your node's identity to wherever it is pointed. The verification rules a collector must implement are in `docs/otel-metrics.md` in the source tree.

19.4 What gets exported, and how your node is identified

Instruments cover transport (bytes, packets, transfers, NAT traversal, RTT, congestion window), operations by type and result, ring connections and lattice neighbors, hosted contracts by reason and bytes, contract-queue depth and rejections, gateway connect failures, and process RSS (`freenet.process.memory.rss`, Linux only — an empty series on macOS and Windows is expected, not a broken pipeline).

How long your node took to join (since v0.2.145). Two gauges show how quickly a freshly started node found its first peer:

- `freenet.bootstrap.time_to_first_ring_connection_ms`: milliseconds from process start to the node's first ring connection. It is recorded once per run. If the node later loses all its connections and rejoins, the value stays the same, because a rejoin is a reconnect and not a slow first join. There is no datapoint until the first connection exists.
- `freenet.bootstrap.connect_attempts_before_first_connection`: the number of join (CONNECT) attempts the node made before that first connection. It is reported from startup, stops counting once the node is connected, and then keeps its final value.

These gauges tell you whether a slow start came from slow first contact (high time, many attempts) or from slowly filling out the rest of the connections. That second part is what `freenet.bootstrap.completed` tracks. Gateways never get a time-to-first-connection value, because a gateway's "first connection" is just whenever someone dials it, and their attempts gauge stays at 0. The project telemetry (`telemetry-enabled`) carries the same two numbers as `bootstrap_time_to_first_ring_connection_ms` and `bootstrap_connect_attempts_before_first_connection`. Neither contains peer addresses.

Every batch carries two resource attributes: `freenet.node.pubkey` (your node's transport public key) and `freenet.node.fingerprint` (the short form shown in the dashboard). **Both are sent in every auth mode, `disabled` included** — `disabled` withholds the signature, not the identity, because metrics with no node id to group them by would be useless. If you do not want a node identifiable to a collector, do not export to that collector.

Export failures never affect the node: a collector that is down produces a warning naming the endpoint and reason — once per failing streak, not once per interval — and an informational line when exports recover.

Part VIII — Uninstalling

Freenet removes cleanly:

```
$ freenet uninstall            # interactive: asks what to do with your data
$ freenet uninstall --keep-data   # remove service + binaries, keep data/config
$ freenet uninstall --purge       # remove everything, including data & logs
```

Add `--system` if you installed the system-wide service. To remove only the service but keep the binary (e.g. switching to hand-run mode), use `freenet service uninstall` instead.

> ⚠️ `--purge` deletes your **secrets** too — any identities or room keys held by delegates are gone permanently unless you exported a bundle first (§16).

Appendices

Appendix A — CLI Quick Reference

Command	One-liner
`freenet` / `freenet network`	Run the node (network mode).
`freenet network --otel-telemetry-enabled --otel-endpoint URL`	Export node metrics to your own OTLP collector (§19).
`freenet local`	Run a sandboxed local-only node.
`freenet --version`	Version + git commit + build timestamp.
`freenet service install [--system] [--no-linger]`	Install as a supervised service.
`freenet service status/start/stop/restart`	Control the service.
`freenet service enable/disable`	Persistently allow/forbid the node to run.
`freenet service logs [--err]`	Follow logs.
`freenet service doctor [--system]`	Repair a wedged install.
`freenet service report [--local PATH] [-m MSG]`	Diagnostic report.
`freenet update [--check] [--force] [--quiet]`	Update the binary; also resets an auto-update lockout (§14.2).
`freenet open -- <link>`	Open a `freenet:` link through your node (what the OS calls for you; §4.2).
`freenet service url-handler register/unregister`	Opt in to or out of `freenet:` link handling (§4.2).
`freenet secrets status/provision/rotate/migrate/snapshots/restore/export/import`	Key & secret management (§16).
`freenet uninstall [--purge | --keep-data] [--system]`	Remove Freenet.
`fdev …`	Developer tool (§18).

Appendix B — Default Ports and File Locations

Ports

Port	Protocol	Scope	Purpose
7509	TCP (HTTP/WebSocket)	localhost only	Dashboard + client API
31337	UDP	internet	Peer-to-peer transport

Directories (defaults; every one is overridable — §6.1)

OS	Config	Data
Linux	`~/.config/freenet/`	`~/.local/share/freenet/`
macOS	`~/Library/Application Support/The-Freenet-Project-Inc.Freenet/`	same tree
Windows	`%APPDATA%\The Freenet Project Inc\Freenet\config\`	`%APPDATA%\The Freenet Project Inc\Freenet\data\`

Binary (installer default): `~/.local/bin/freenet` on Linux/macOS. The macOS app symlinks the CLI tools into `/usr/local/bin`. Docker keeps everything on the `/data` volume (binary at `/data/bin/freenet`).

Appendix C — Exit Codes

Code	Meaning	Action
0	Clean shutdown	none
42	Update requested, or network listener stopped	supervisor runs `freenet update` and restarts; normal during upgrades, otherwise §11.1
43	Another node already running	wrapper auto-resolves orphans; else §11.2
other	Crash	supervisor restarts with backoff; probation crashes may trigger auto-rollback (§13)

Appendix D — Glossary

- **AOF** — append-only file; the node's local event log format.
- **Contract** — WASM program + replicated public state; the unit of shared data on Freenet.
- **Dashboard** — the node's local status/app page at `http://127.0.0.1:7509/`.
- **DEK / KEK** — data-encryption key (per delegate) / key-encryption key (master); see §16.
- **Delegate** — WASM program managing private data, runs only on your node.
- **Gateway** — publicly reachable node that bootstraps new peers.
- **Local mode** — sandboxed single-node mode for development (`freenet local`).
- **Node / peer** — a running `freenet` instance participating in the network.
- **Probation** — the crash-watch window right after an update; see §13.
- **Supervisor** — systemd / launchd / tray wrapper that restarts and updates the node.
- **WS API** — the node's local WebSocket client API (port 7509).

Appendix E — Manual Revision History & Growth Chart

The manual grows with the software and with its readers. Each row records a revision; the chart shows how coverage has expanded. New and updated sections in the *current* revision are additionally badged inline throughout the text.

Manual rev	Date	Freenet version	What changed
1.19	2026-11-09	0.2.156	A bad gateway entry or a dead gateway index no longer stops startup (built-in gateways as a fallback); bandwidth limits now hold on QUIC and are shared fairly under a node-wide cap, and `total-bandwidth-limit = 0` no longer quietly throttles; nodes on `prefer` now carry data over QUIC, so a crash-breaker trip is no longer free for them; desktop programs can use an app's delegate only after you approve it.
1.18	2026-11-04	0.2.155	QUIC shadow mode is now on by default on Linux and macOS (rev 1.17's "off by default" was true only up to v0.2.154), with a new Windows short-run breaker; apps can no longer impersonate the node to a delegate; the macOS 26 menu bar icon shows again; loopback API no longer resolves the hostname at startup.
1.17	2026-11-03	0.2.149–0.2.154	Catch-up over six releases: a node that cannot download an update now stays up on a cooldown instead of restart-looping, with a daily download budget; `bandwidth-limit` actually works since v0.2.152, with its real default stated; the experimental QUIC switches and their risks; delegates on hosted nodes scoped to their user; `fdev verify-merge` checks newest states first.
1.16	2026-10-23	0.2.148	`total-bandwidth-limit` is now a real cap on everything your node uploads, with what to do when transfers stall under a low cap; a malformed packet from a peer no longer panics the connection.
1.15	2026-10-19	0.2.147	Apps left open no longer stop receiving live updates after 30 minutes; saves no longer hang for minutes when a quick check's reply is lost; `fdev verify-merge` and the dashboard stop flagging capped contracts (chat rooms, feeds) as associativity violations.
1.14	2026-10-15	0.2.146	Windows nodes no longer restart themselves on routine network errors (and exit code 42 is explained honestly); a new §11.8 for an app that keeps showing its old version, now fixed for app pages; apps opened during startup no longer go quiet for minutes.
1.13	2026-10-14	0.2.145	A one-minute live bandwidth chart on the dashboard; app links opened while the node is still joining now wait and open, instead of being dropped on the dashboard; a node silently stuck on loopback by an old `freenet local` run fixes itself; two new join-time metrics for your own collector.
1.12	2026-10-12	0.2.144	The Linux service now clears a leftover node on the same version too, instead of standing down beside it; the dashboard names the disk "Disk used" leaves out — the database file and its dead space, and the shared webapp cache.
1.11	2026-10-10	0.2.143	The node downloads an update before restarting, ending an endless restart loop on slow links; the network now refuses v0.2.120 and v0.2.121, which cannot update themselves; diagnostic reports go to `telemetry.freenet.org` and work behind TLS-inspecting proxies (updates still don't); the peer page says so when it sees no difference between peers.
1.10	2026-10-06	0.2.142	`FREENET_ROUTING_HIERARCHICAL` no longer does anything — the old router it could select is gone, and the manual's "set to 0 to fall back" was now false; the fallback-sounding variable that is *not* a replacement; peer pages redesigned around what the node learned, with the skill score moved to a new `/routing` page.

Manual rev	Date	Freenet version	What changed
1.9	2026-10-04	0.2.141	Hosting RAM is now measured rather than charged at 1 MiB per contract, under a `--hosting-mem-share` dial this manual had never documented; a correction to §17, which had claimed live measurement since v0.2.126; the dashboard stops painting normal states red.
1.8	2026-09-30	0.2.140	`freenet:` links open apps through your own node (new §4.2), with how they are registered and what a hostile link cannot do; a NixOS module and its `services.freenet` name trap; the auto-update lockout — now documented at all, and no longer permanent.
1.7	2026-09-28	0.2.139	Periodic wake-ups: a granted app can now also run on a schedule, under the same permission re-checked at every fire and the same budget, bounded by the node at 60 s–7 days and four schedules per delegate; the permissions page gains a way back to the dashboard.
1.6	2026-09-25	0.2.138	App permissions: a delegate may ask to run with no tab open, the node (not the app) asks once, the answer belongs to the app, "Not now" holds for a week, and `/permission/apps` revokes; delegate manifests for authors; a stranded-transfer race fixed.
1.5	2026-09-24	0.2.137	Delegate subscriptions survive a node restart (with a warm-up and two ceilings); a missing app asset answers 404 instead of a 500 that leaked an OS error, and a new §11.7 for the case only the log can explain; typed `DelegateError::Missing` for network-mode clients.
1.4	2026-09-22	0.2.136	Nix as a supported deployment path; per-peer dashboard pages; fullscreen apps; probation announcements that can't cause the rollback they report; and two corrections to this manual's own storage-tuning claims.
1.3	2026-09-11	0.2.135	Update-safety warnings when crash-loop rollback is not armed; deliberate stops no longer counted as crashes; exact `config.toml` honored; SELinux/user-service install fixes; delegates reach the network under a 256-subscription cap; stdlib 0.10.0.
1.2	2026-09-07	0.2.134	Metrics export to your own OpenTelemetry collector (new §19); project telemetry endpoint moved to `telemetry.freenet.org`; corrected Matrix room; role-based gateway names; app-visible reliability fixes; credential redaction in diagnostic reports.
1.1	2026-09-05	0.2.133	First living revision: Docker install, macOS app, version-floor warning, bounded logs, memory-aware budgets, dashboard growth, `fdev verify-merge`, backup guidance. Full delta ledger below.
1.0	2026-08-25	0.2.123	Initial full manual: concepts, install, operations, ten-step self-check, troubleshooting, auto-update & rollback, secrets, tuning, developer intro, appendices.

Revision 1.19 delta ledger (every badge in *this* edition traces to a row here; "driver" names the upstream release or marks the change as editorial):

Section	Badge	Change	Driver
§3 First run	UPDATED	First-start gateway fetch falls back to the cache plus the gateways built into the binary	v0.2.156
§4.1 Background apps	UPDATED	User-approved grants let native (desktop) programs use an app's delegate, revocable, at most 64; hosted visitors are told where their data is kept and how to install Freenet	v0.2.156
§6.2 Options	UPDATED	QUIC paced at the bandwidth limits; one fair queue under `total-bandwidth-limit` with no isolation between connections; `total-bandwidth-limit = 0` fixed; long broadcasts no longer resent after a needless timeout; `prefer` nodes carry data over QUIC (release share still 0)	v0.2.156
§11.4 No peers	UPDATED	Bad gateway entries and unresolvable addresses are skipped with a warning; built-in fallback gateways when the index is down; startup fails only when no gateway is usable	v0.2.156

Revision 1.18 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§2.3 Desktop apps	UPDATED	macOS 26: the menu bar icon shows again; an old Launch at Login entry is repaired by itself	v0.2.155
§4.1 Background apps	UPDATED	Clients may send delegates only application messages; node-only kinds are refused with an error	v0.2.155
§6.2 Options	UPDATED	Unset `transport-quic` now means `shadow` on Linux and macOS (`off` elsewhere), so the crash breaker is armed by default there; Windows short-run breaker; `allowed-host` on a loopback bind no longer adds the hostname's addresses	v0.2.155

Revision 1.17 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§4.1 Background apps	UPDATED	Delegate unregistration, context and hosted runs are scoped to their user on hosted nodes; tokenless clients there now get 403	v0.2.150
§6.2 Options	UPDATED	Streams served in order under `total-bandwidth-limit`; `bandwidth-limit` kept on every rate update and its real default (1,250,000 B/s) stated; one paragraph on the off-by-default QUIC switches, `quic-max-paths`, `quic-data-percent` and the 24 h crash breaker	v0.2.149, v0.2.152
§13 Auto-update	UPDATED	A failed release download puts the release on a cooldown (≈1 h, then ≈2 h); the third failure in a row exits 42 for the updater; 320 MiB per 24 h download budget; `freenet service report` status line	v0.2.149
§14.2 After a bad update	UPDATED	The lockout's daily retry also waits for the cooldown and the download budget	v0.2.149
§18 Developer	UPDATED	`fdev verify-merge` checks newest states first, reports what it skips, bounded `--max-states-paired`	v0.2.153

Revision 1.16 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§6.2 Options	UPDATED	`total-bandwidth-limit` documented for the first time: a node-wide egress cap on all UDP traffic (opt-in, default unset); `min-bandwidth-per-connection` is a floor inside it; 0 is ignored; stall warnings and their remedies	v0.2.148
§11.3 Stuck on an old version	UPDATED	Malformed or undecodable peer packets no longer panic the receive path; the one-per-connection warning, and what it suggests when many peers trigger it	v0.2.148

Revision 1.15 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§4 Using applications	UPDATED	App push registrations live as long as the tab's connection (no 30-minute cutoff; 128-tab cap per delegate behind a reverse proxy); a lost summary-first PUT probe now times out in ≤ ~15 s instead of ~1.5–2.5 min	v0.2.147
§18 Developer	UPDATED	`fdev verify-merge` / "Merge laws" card: associativity reported only when the two groupings do not converge within 4 heal passes; earlier flags on capped contracts likely false positives	v0.2.147

Revision 1.14 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§4.2 `freenet:` links	UPDATED	Subscriptions made through the bootstrap gateway are renewed through the ring once joined (retry every 2 min), so an app opened at startup no longer goes quiet for several minutes	v0.2.146
§8 Reading the signals	UPDATED	Exit code 42 also means "network listener stopped", not only "update me"	v0.2.146
§11.1 Keeps stopping	UPDATED	Routine receive errors (e.g. Windows WSAEMSGSIZE/WSAENETRESET) no longer stop the listener; the log lines, the rate limit, and when it still exits	v0.2.146
§11.8 Old app version	NEW	App HTML served with `Cache-Control: no-store`, so a normal reload gets the current version; assets are not covered — hard reload	v0.2.146
Appendix C Exit codes	UPDATED	Exit 42 row matches §8	v0.2.146

Revision 1.13 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§3 Dashboard	UPDATED	Data Transfer card gains "Transfer rate, last minute": 60 s of upload/download at 1 s resolution, "as of N s ago" when stale, local-only	v0.2.145
§4.2 `freenet:` links	UPDATED	An app link opened while the node is connecting stays on the app's address and retries for up to 2 minutes instead of falling to the dashboard after 3 s	v0.2.145
§11.4 No peers	UPDATED	New check: a stored loopback `network-address` left by `freenet local` is now ignored and rewritten on non-gateway nodes; how to keep a deliberate loopback bind	v0.2.145
§19.4 OTel export	UPDATED	Two bootstrap gauges: time to first ring connection, and join attempts before it (gateway behaviour stated)	v0.2.145

Revision 1.12 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§11.2 Port already in use	UPDATED	systemd start-up check now stops an orphaned `freenet network` of the same user whatever its version (separately configured instances and other services' nodes spared); existing units refreshed by the next update	v0.2.144
§11.6 Disk usage growing	UPDATED	"Disk used" tooltip lists the measured database file (with its excess over state) and the shared webapp cache, neither of which counts against the disk budget	v0.2.144

Revision 1.11 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§3 Dashboard	UPDATED	Peer page states "No measurable difference between your peers' …" instead of charting a ranking the node has not learned	v0.2.143
§10 Diagnostic reports	UPDATED	Reports upload to `telemetry.freenet.org`; report upload alone also trusts the OS certificate store, so it works behind TLS-inspecting proxies while updates do not; failed uploads now name the `--local` fallback	v0.2.143
§13 Auto-update	UPDATED	Release downloaded and cached before exit 42 (up to 2 h, resumable, re-verified at install), fixing the slow-link restart loop; `min-compatible-version` floor raised to v0.2.122, refusing v0.2.120–0.2.121	v0.2.143

Revision 1.10 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§3 Dashboard	UPDATED	Peer pages redesigned around what the node learned about each peer versus distance alone and versus other peers; network-wide prediction quality, including the skill score, moved to new `/routing`. Replaces this section's rev 1.4 description of the skill score as the peer page's headline	v0.2.142
§6.2 Options	UPDATED	`FREENET_ROUTING_HIERARCHICAL` is obsolete and inert (legacy routing stack removed), replacing the now-false "set to 0 to fall back"; warning that `FREENET_ROUTING_FALLBACK_ISOTONIC` is an emergency fallback, not the old router	v0.2.142

Revision 1.9 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§3 Dashboard	UPDATED	Normal states no longer painted as alarms: the failure count is no longer red at 0, "next to evict" no longer orange far under budget, and a full cache reads "Full is normal here"	v0.2.141
§6.2 Options	UPDATED	`--hosting-mem-share` added: share of the node's memory limit hosting may hold in RAM, default ⅛, floor 64 MiB, warning above 0.5	v0.2.141; editorial (omission)
§17 Tuning	UPDATED	RAM side of hosting measured rather than estimated at 1 MiB per contract, sized against the cgroup-aware memory limit, independent of the disk dial; corrects this section's earlier claim of live measurement since v0.2.126	v0.2.141; editorial (correction)

Revision 1.8 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§2.7 Nix	UPDATED	NixOS module `services.freenet-node` and its options; the trap that `services.freenet` is nixpkgs' alias for Hyphanet; keep the service user's `home`, or crash-loop rollback is silently off; `STATE_DIRECTORY` fallback and the manual-update invocation that needs it	v0.2.140
§4.2 Opening `freenet:` links	NEW	Link handler registration per platform (never replaces a handler you chose), opt-out that sticks, and what a hostile link cannot do: fixed local host and port, no shell, never starts a stopped node	v0.2.140
§14.2 After a bad update	UPDATED	The auto-update lockout after three consecutive failures, previously undocumented here and previously permanent, now retries at most daily; `freenet update` resets it; the usual cause	v0.2.140; editorial (omission)
Appendix A	UPDATED	`freenet open`, `freenet service url-handler register/unregister`, and `freenet update` as the lockout reset	v0.2.140

Revision 1.7 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§3 Dashboard	UPDATED	The Apps and permissions page gains a link back to the dashboard	v0.2.139
§4.1 Background permission	UPDATED	A granted app may also run on a periodic schedule — same grant re-checked at every fire, same budget, node-imposed bounds (60 s–7 days, four schedules per delegate, one pending fire each), deferral rather than backlog, and nothing carried across a restart	v0.2.139
§18 Developer	UPDATED	`wakeups = [tag = seconds]` in the delegate manifest and `WakeupFired`; why it is declared rather than requested (forward compatibility with older nodes); the bounds to build against; `freenet-stdlib` 0.12.1	v0.2.139

Revision 1.6 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§3 Dashboard	UPDATED	New *Apps and permissions* card linking to `/permission/apps`, where grants are listed and revoked	v0.2.138
§4 Using applications	UPDATED	A reordering race that stranded large transfers is fixed — both fragment/metadata orderings now handled	v0.2.138
§4.1 Background permission	NEW	The node-authored consent-once prompt: granted to the app rather than the delegate, so a delegate update does not re-ask; "Not now" holds for seven days; a dismissal stores nothing; budgets bound unprompted runs; and an explicit statement of what the permission does *not* defend against	v0.2.138
§18 Developer	UPDATED	Delegate manifests (`#[delegate(manifest(...))]`), the `Background` capability, lifecycle events, and the three limits to build against: 8 apps per delegate, budgeted unprompted runs, 64 KiB parameter ceiling for lifecycle delivery	v0.2.138

Revision 1.5 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§4 Using applications	UPDATED	Delegate subscriptions survive a restart: persisted and replayed at startup, after a ~15 s warm-up, capped at 4096 stored and 512 re-established per boot, with no migration across a delegate upgrade	v0.2.137
§5 The Service	UPDATED	A restart no longer leaves delegates unsubscribed from the contracts they were following	v0.2.137
§11.7 App loads but parts are missing	NEW	A missing app asset now answers 404 with `Cache-Control: no-store` instead of a 500 carrying an OS error; the permissions/damaged-cache case is distinguishable only from the node's log warning	v0.2.137
§18 Developer	UPDATED	Typed `DelegateError::Missing(key)` for network-mode clients in place of a stringly generic `OperationError`; missing-delegate probes no longer logged at error level	v0.2.137

Revision 1.4 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§2.7 Nix	NEW	Nix as a supported deployment path: `freenet-node` self-updates, `freenet` is build-only, and the `overlays.default` trap that puts a non-updating binary on every user's PATH	v0.2.136
§3 Dashboard	UPDATED	Per-peer detail pages with routing-prediction skill scores, negative values labelled in words	v0.2.136
§4 Using applications	UPDATED	Apps may go fullscreen; streaming relay failures reported upstream instead of going silent	v0.2.136
§6.2 Options	UPDATED	`--max-hosting-storage` default corrected to memory ÷ 8 (1 GiB is the cap); `FREENET_ROUTING_HIERARCHICAL` on by default	v0.2.136
§11.2 Port already in use	UPDATED	The single-instance check retries past a dying process's socket, so an immediate restart after a stop no longer fails	v0.2.136
§13 Auto-update	UPDATED	End-of-probation announces the actual outcome, one wording per outcome, and can no longer panic — a failed write could previously have triggered the rollback it was reporting as unnecessary	v0.2.136
§17 Tuning	UPDATED	Two corrections to this manual: the default is memory-derived, not a flat 1 GiB, and the dial governs disk rather than RAM. Adds the headroom rule and a worked example	v0.2.136
§18 Developer	UPDATED	Synchronous delegate contract-write host functions removed — a breaking change for delegates that write contract state	v0.2.136

Revision 1.3 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§2.2 Recommended install	UPDATED	SELinux file contexts restored after install; a user-level binary now gets a user-level service	v0.2.135
§4 Using applications	UPDATED	Delegates can GET and SUBSCRIBE over the network; 256-subscription cap evicts the least-recently-notified rather than refusing	v0.2.135
§5 The Service	UPDATED	A deliberate `service stop` / `systemctl stop` is no longer counted as a crash by the rollback detector	v0.2.135
§6.1 Where things live	UPDATED	Exact `config.toml` is loaded directly; the directory is no longer scanned, so a `config.bak.toml` can't be picked up instead	v0.2.135
§13 Auto-update	UPDATED	The two states where crash-loop rollback protection is silently off now warn, and `--quiet` does not suppress them	v0.2.135
§14.2 After a bad update	UPDATED	Check whether rollback was armed before assuming it will save you; manual recovery steps	v0.2.135
§16 Secrets	UPDATED	Delegate WASM gains the wall-clock backstop and panic capture contracts had; delegates parked off the serial loop; secrets decrypted once per read	v0.2.135
§18 Developer	UPDATED	Delegate network GET/SUBSCRIBE, the non-configurable 256 cap and its eviction rule, `freenet-stdlib` 0.10.0	v0.2.135

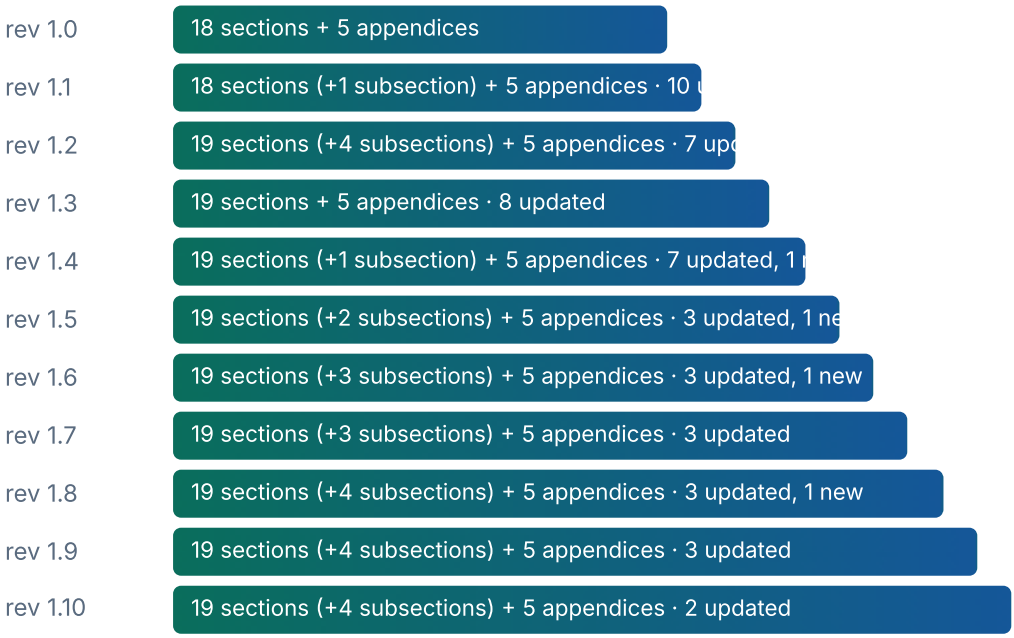
Revision 1.2 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

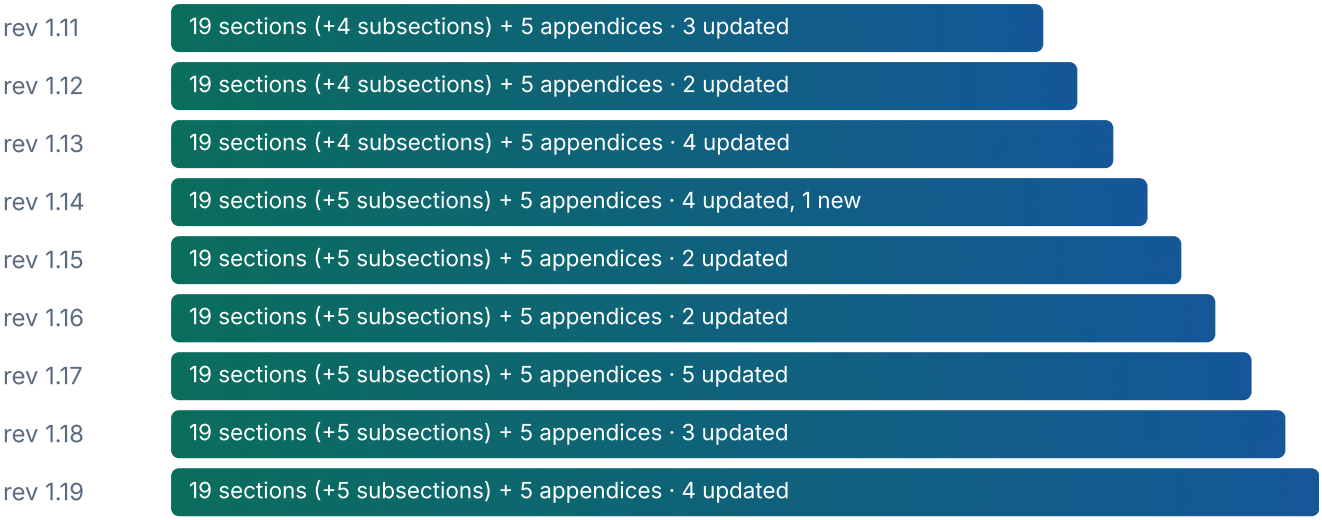
Section	Badge	Change	Driver
§1.3 Where to learn more	UPDATED	Community chat is `#freenet-locutus:matrix.org`; the previously listed room was not the project's	v0.2.134
§4 Using applications	UPDATED	Streaming PUTs no longer misreported as failed; locally-stored contracts not suppressed by a transient fetch failure; delegate failures surfaced to the app and delegates notified on initial state install	v0.2.134
§6.2 Options	UPDATED	Telemetry endpoint default moved to `telemetry.freenet.org:4318` with automatic migration of the old value; `--otel-telemetry-enabled` added	v0.2.134
§6.4 Running a gateway	UPDATED	Gateways named by role (`gw1` / `gw2.freenet.org`) in examples, keys, and docs	v0.2.134
§10 Diagnostic reports	UPDATED	Echoed credentials and URL userinfo redacted from logs before a report uploads them	v0.2.134
§12 Getting help	UPDATED	Corrected Matrix room	v0.2.134
§19 Monitoring	NEW	Full section: OTLP metrics exporter — enabling it, endpoint precedence, cleartext-credential refusal, signed-token auth mode, exported instruments and node identity	v0.2.134
Appendix A	UPDATED	Metrics-export invocation added to the quick reference	v0.2.134

Revision 1.1 delta ledger (historical — these badges are no longer shown inline; kept so each edition's changes stay on the record):

Section	Badge	Change	Driver
§2.3 Desktop apps	UPDATED	macOS DMG app with automatic CLI `PATH` setup; Windows binaries Authenticode-signed	v0.2.125/0.2.133; v0.2.131
§2.6 Docker	NEW	Official self-updating container image `ghcr.io/freenet/freenet-core`	v0.2.133
§3 Dashboard	UPDATED	Measured GET success rate, `/contract/{key}` detail pages, OS theme, filterable tables	v0.2.129–0.2.130
§7 Health check	UPDATED	Step 7 warning: compatibility floor makes staying current mandatory	v0.2.133
§11.4 No peers	UPDATED	New first cause: node below the network's minimum compatible version	v0.2.133
§11.6 Disk usage	UPDATED	Logs auto-pruned to 512 MiB, `FREENET_LOG_DIR_MAX_BYTES` override; WASM cache disk-bounded	v0.2.130; v0.2.126
§13 Auto-update	UPDATED	`min-compatible-version` enforced as a hard gate at the transport handshake	v0.2.133
§16 Secrets	UPDATED	Backup-as-a-habit guidance (secrets are the only unrecoverable data)	editorial
§17 Tuning	UPDATED	Live-memory-aware hosting budgets, ~2 GiB peer memory cap, disk-headroom-bounded WASM cache	v0.2.126–0.2.127
§18 Developer	UPDATED	`fdev verify-merge` (merge-law verifier, formerly `conformance`)	v0.2.129–0.2.133
Appendix B	UPDATED	macOS `/usr/local/bin` symlinks; Docker `/data` volume paths	v0.2.133

Coverage growth chart (sections present per revision):





Reading the chart: each future revision adds a row; the bar length is proportional to the number of sections, so the chart literally shows the manual — and the platform — growing. Compare any two revisions via the badges listed in their rows to see exactly what to re-read.
